

UNIVERSIDAD VERACRUZANA

FACULTAD DE ESTADÍSTICA E INFORMÁTICA

# REVISIÓN SISTEMÁTICA DE LA LITERATURA (RSL): “PRUEBAS ORIENTADAS A SISTEMAS BASADOS EN LA ARQUITECTURA DE MICROSERVICIOS”

MODALIDAD:
MONOGRAFÍA

QUE PARA OBTENER EL GRADO DE:
LICENCIADO EN INGENIERÍA DE SOFTWARE

PRESENTA:
SEBASTIÁN BELLO TREJO

DIRECTORES:
DR. HÉCTOR XAVIER LIMÓN RIAÑO
DR. JORGE OCTAVIO OCHARÁN HERNÁNDEZ

XALAPA, VER. 15 DE DICIEMBRE 2023

# CONTENIDO


1 RESUMEN ..... 2

2 INTRODUCCIÓN ..... 3

3 ANTECEDENTES ..... 5

4 MÉTODOS ..... 8

4.1 PREGUNTAS DE INVESTIGACIÓN ..... 9
4.2 ESTRATEGIA DE BÚSQUEDA ..... 10
4.2.1 TÉRMINOS DE BÚSQUEDA ..... 11
4.2.2 CADENAS DE BÚSQUEDA ..... 11
4.2.3 SELECCIÓN DE FUENTES ..... 15
4.3 SELECCIÓN DE ESTUDIOS PRIMARIOS ..... 16
4.3.1 CRITERIOS DE SELECCIÓN DE ESTUDIOS PRIMARIOS ..... 16
4.3.2 PROCEDIMIENTO DE SELECCIÓN DE ESTUDIOS PRIMARIOS ..... 17
4.4 EXTRACCIÓN DE DATOS ..... 17
4.5 ESTRATEGIA PARA LA SÍNTESIS DE DATOS ..... 18
4.5.1 DESARROLLO DE UNA TEORÍA ..... 18
4.5.2 DESARROLLO DE UNA SÍNTESIS PRELIMINAR ..... 18
4.5.3 EXPLORACIÓN DE LAS RELACIONES DENTRO Y ENTRE ESTUDIOS ..... 19
4.5.4 EVALUACIÓN DE LA ROBUSTEZ DEL PRODUCTO DE LA SÍNTESIS ..... 19
4.6 CONDUCCIÓN DE LA REVISIÓN ..... 19
4.6.1 HERRAMIENTAS ..... 20

5 RESULTADOS ..... 25

5.1 PRUEBAS ORIENTADAS A SISTEMAS BASADOS EN LA ARQUITECTURA DE MICROSERVICIOS ..... 26
5.1.1 CONTEXTO DE APLICACIÓN DE PRUEBAS ..... 34
5.2 PROPUESTAS DE LA LITERATURA ..... 34
5.3 HERRAMIENTAS ..... 44
5.4 RETOS ..... 51

6 CONCLUSIONES ..... 55

7 BIBLIOGRAFÍA ..... 56

8 REFERENCIAS ..... 59

9 APÉNDICES ..... 65

# 1 Resumen

Un microservicio es un servicio pequeño que corre en su propio proceso y comunica con otros por medio de mecanismos ligeros, comúnmente interfaces de programación de aplicaciones (API, por sus siglas en inglés) (Fowler y Lewis, 2014). Asimismo, cada servicio desempeña una función de negocio (Alshuqayran et al., 2016, p. 44), produciendo servicios débilmente acoplados y altamente cohesivos (Gong y Cai, 2023, p. 141). Bajo estos principios surge el estilo arquitectónico de microservicios, el cual beneficia características como la mantenibilidad, escalabilidad o tolerancia a fallos por parte de un sistema (Wu et al., 2022, p. 467). Sin embargo, el estilo también cuenta con distintos retos técnicos, económicos e incluso psicológicos (Larrucea et al., 2018, p. 99) presentados a lo largo del ciclo de vida de desarrollo de un producto de software; como en el caso de las pruebas en microservicios (Bogner et al., 2021, pp. 19-20). En comparación con otras arquitecturas, probar microservicios presenta numerosas dificultades a causa de la complejidad de las dependencias de los servicios y la heterogeneidad de los ambientes de ejecución (Wang et al., 2020, p. 2; Ding et al., 2020, p. 188) o la intricada optimización de pruebas manuales (Elsner et al., 2022, p. 4).

Además de estos retos, otro de los obstáculos de probar microservicios es la dispersión de información en la literatura. Ya que como aún se percibe una brecha notoria entre la industria y la academia (Alshuqayran et al., 2016, p. 48), se pasan por alto propuestas valiosas que buscan resolver varios de los problemas inherentes al estilo arquitectónico de microservicios. Aun cuando ya se cuenta con un trabajo con un propósito similar al de este (Waseem et al., 2020), es requerida una actualización de la información recabada. Por ello, el presente trabajo tiene el objetivo de recopilar información relacionada con pruebas orientadas a sistemas basados en la arquitectura de microservicios, ofreciendo un análisis del estado del arte. Con la finalidad de brindar información a investigadores y practicantes de ingeniería de software interesados en el desarrollo de microservicios.

Como método se llevó a cabo una revisión sistemática de la literatura (RSL) con base en la propuesta de Kitchenham et al. (2015). La estrategia de búsqueda fue realizada con apoyo del enfoque sistemático de Zhang et al. (2011), obteniendo una selección de 53 estudios primarios publicados entre enero de 2018 y noviembre de 2023, a través de los motores de búsqueda IEEE Xplore, ACM Digital Library, SpringerLink y ScienceDirect. Posteriormente, la síntesis de los estudios se atuvo al método narrativo sugerido por Rodgers et al. (2009). Entre los hallazgos se encontraron propuestas para la detección de fallos y la pérdida de características de calidad en la operación de servicios, como lo son el rendimiento o la fiabilidad. Asimismo, es perceptible la preocupación de la industria por solucionar los retos u obstáculos asociados al estilo arquitectónico. Finalmente, dada la naturaleza de los microservicios, la modificación de las estrategias de prueba tradicionales es necesaria para satisfacer las necesidades de este estilo arquitectónico.

## 2 Introducción

La dificultad de probar sistemas basados en la arquitectura de microservicios está dada por obstáculos como la complejidad de las dependencias entre servicios, la heterogeneidad de los ambientes de operación, lo conflictivo que resulta localizar defectos, la ineficacia de los estándares de prueba de software y modelos de calidad actuales (Gong y Cai, 2023, p. 141), la naturaleza asíncrona con la que interactúan los microservicios (Zhou et al., 2018, p. 243), la disparidad de resultados de las pruebas de un mismo servicio en diferentes ambientes o el alto costo de la verificación al utilizar métodos tradicionales de prueba (Li et al., 2020, p. 363-364).

Hasta 2014, Fowler y Lewis consideraban que no existía mucha información sobre cómo desarrollar microservicios correctamente. No obstante, en los últimos años se han dedicado esfuerzos de investigación importantes centrados en microservicios. Por ejemplo, en el caso de las pruebas de microservicios han surgido metodologías y herramientas (Newman, 2015) que contribuyen a intentar resolver varios de los problemas intrínsecos a los microservicios. Particularmente, en la literatura se encuentran diferentes propuestas que atienden varias de las limitaciones de las metodologías o herramientas de facto de la industria. Desafortunadamente, dado que existe una brecha entre la academia y la industria (Alshuqayran et al., 2016, p. 48), lo que se propone en la academia suele ignorarse en la práctica. Por estas razones, el objetivo de este trabajo es recopilar información relacionada con pruebas en sistemas basados en la arquitectura de microservicios, con la finalidad de ponerla a disposición de investigadores y practicantes de ingeniería de software interesados en microservicios. Lo cual resulta significativo, si consideramos que se estimaba que, a partir de 2021, el 80% de los desarrollos de aplicaciones en la nube implementarían microservicios (Larrucea et al., 2018, p. 96).

Buscando analizar el estado del arte, se propusieron las siguientes preguntas de investigación ($RQ$) que contribuyen al objetivo de esta revisión:

$RQ$-1: ¿Cuáles son las pruebas de software dedicadas a sistemas basados en la arquitectura de microservicios?
$RQ$-1.1: ¿En qué contexto se aplicarían las distintas pruebas orientadas a la arquitectura de microservicios?

La identificación de las distintas pruebas utilizadas en sistemas basados en la arquitectura de microservicios y su catalogación asistiría a la planeación y diseño de estrategias de prueba.

$RQ$-2: ¿Cómo prone la literatura abordar el proceso de prueba orientado a sistemas basados en la arquitectura de microservicios?

La detección de propuestas de la literatura centradas en pruebas dirigidas a la arquitectura de microservicios facilitaría la disposición de información a

investigadores y practicantes de ingeniería de software interesados en el desarrollo de microservicios.

$RQ$-3: ¿Cuáles son las herramientas utilizadas para probar sistemas basados en la arquitectura de microservicios?

Ya que se necesita contar con tanta confianza como sea posible durante la etapa de pruebas (Fowler y Lewis, 2014), la implementación de herramientas de software se convierte en un requisito indiscutible. Por ello, es importante identificar tanto las herramientas propuestas por la literatura, como aquellas utilizadas en la industria.

$RQ$-4: ¿Cuáles son los retos existentes asociados al proceso de prueba en la arquitectura de microservicios?

Entendiendo que los métodos tradicionales de prueba enfrentan diferentes dificultades cuando se aplican sobre la arquitectura de microservicios (Gong y Cai, 2023, p. 141). Se precisa analizar el estado actual de las pruebas en microservicios, buscando que futuros trabajos de investigación resuelvan varios de los problemas inherentes al proceso de prueba de microservicios.

# 3 Antecedentes

Durante un simposio sobre computación en la nube en 2005, Peter Rogers acuñó el concepto “micro web service” (Kalubowila, 2021, p. 49), pero no fue hasta mayo de 2011, durante un taller de arquitectos de software en Venecia, el momento en el que surgiese el término “microservice” (Fowler y Lewis, 2014). Actualmente, se conoce que un microservicio es un servicio pequeño el cual cuenta con una responsabilidad única; tornando independiente su despliegue, escalamiento y pruebas (Thönes, 2015, p. 116). Esto posibilita que cada microservicio se construya a partir de una capacidad o función de negocio específica (Fowler y Lewis, 2014) dentro de un contexto limitado (Richards y Ford, 2021a, p. 246). Dotando a las organizaciones con la habilidad de mejorar sus tiempos de producción e incrementar las entregas continuas (Larrucea, 2018, p. 96).

La arquitectura de software se define como el conjunto de estructuras las cuales integran elementos de software (componentes e interfaces), las relaciones entre ellos y los propiedades de ambos (Bass et al, 2015, p. 4). La arquitectura de un sistema también puede considerarse como la comprensión compartida de su diseño, específicamente sobre cómo el sistema se divide en componentes y cómo estos se comunican mediante interfaces (Fowler, 2003, pp. 11-12). Un estilo arquitectónico describe la relación nominal entre componentes, los cuales cubren una variedad de características arquitectónicas tanto benéficas como perjudiciales. Generalmente, el concepto de estilo arquitectónico tiende a confundirse con lo que es un patrón arquitectónico, no obstante, aunque son elementos complementarios, se trata de dos ideas totalmente distintas. Un patrón arquitectónico es una estructura de diseño de bajo nivel que brinda una solución a un problema inherente a un estilo arquitectónico. (Richards y Ford, 2021b, pp. 117-119).

El estilo arquitectónico de microservicios surgió como una variante de la arquitectura orientada a servicios (SOA, por sus siglas en inglés) en respuesta a las constantes frustraciones que enfrentaban arquitectos y desarrolladores de software con relación al condicionamiento de la rápida innovación y entrega ágil (Li et al., 2020, p. 363), el fuerte acoplamiento (Jiang et al., 2022, p. 2339), el alto costo de la escalabilidad o la complejidad del mantenimiento (Fowler y Lewis, 2014) por parte de arquitecturas tradicionales. La diferencia entre la arquitectura orientada a servicios y los microservicios, es que la última se enfoca la aplicación de prácticas de desarrollo DevOps orientadas a la entrega continua, la utilización de infraestructura automatizada, así como la descentralización de la gobernanza y gestión de datos (Francesco et al., 2017, p. 21).

Por otra parte, la prueba de software es el conjunto de actividades que forma parte del amplio proceso de verificación y validación de software, las cuales buscan demostrar al desarrollador y cliente que el software satisface los requisitos especificados, así como identificar defectos (Sommerville, 2016, p. 228). Esto es importante ya que, en gran medida, la identificación de defectos permite robustecer el comportamiento de un sistema, pues este debe prevenir cualquier anomalía que

se presente durante su operación o interacción con el usuario. Por ello, se considera que al probar software se busca demostrar la presencia de errores y no su ausencia (Dijkstra, 1972, p. 864).

Las pruebas de software tienden a clasificarse como pruebas funcionales, pruebas no funcionales, pruebas de caja de negra y pruebas de caja negra. Mientras que los niveles de prueba comúnmente se dividen en pruebas de unidad, pruebas de integración o componentes, pruebas de sistema y pruebas de aceptación (International Software Testing Qualifications Board [ISTQB], 2023, p. 15). Aunque existen otros niveles de prueba como los son las pruebas en la nube, contractuales, punto-a-punto o exploratorias, cuya implementación depende del grado de interacción entre componentes que requiera un sistema. Finalmente, es importante entender que la progresión de un nivel de prueba a otro busca expandir el alcance del esfuerzo de prueba, pero reducir el número de casos de prueba (Clemson, 2014, p. 24).

Los métodos y prácticas de las pruebas se dividen en cuatro actividades generales: prueba estática, prueba dinámica, demostración y análisis de la verificación y validación. La prueba estática consiste en la evaluación de un ítem de prueba en tanto no necesite la ejecución de código fuente, la cual puede ser aplicada manualmente o por medio de herramientas. Por otro lado, la prueba dinámica involucra la ejecución de código para realizar pruebas sobre este, las cuales pueden ser aplicadas en cualquier punto del ciclo de vida de software (International Organization for Standardization [ISO] et al., 2022, pp. 17-18). En el caso del análisis de la verificación y validación, su objetivo es ayudar a establecer confianza de que un sistema de software se adapta al propósito por el que fue construido (Sommerville, 2016, p. 228).

La verificación es el proceso de evaluación de un sistema o componente para determinar si el producto de una fase de desarrollo satisface las condiciones especificadas, mientras que la validación tiene como propósito evaluar un sistema o componente a lo largo de su fase de desarrollo para determinar si satisface los requisitos especificados (Institute of Electrical and Electronics Engineer [IEEE], 2017, p. 26). En esencia, la validación ayuda a responder si se está construyendo el producto de software correcto, y la verificación busca asegurar que el producto en desarrollo se esté construyendo correctamente (Sommerville, 2016, p. 228). La unión de ambos procesos, denominada V&V, ofrece una evaluación objetiva de los productos y procesos a lo largo del ciclo de vida de un sistema de software (IEEE, 2017, p. 18).

Como antecedente del presente trabajo, Waseem et al. (2019) publicaron un mapeo sistemático que contribuye al estado del sobre los enfoques de prueba de aplicaciones basadas en la arquitectura de microservicios. Este estudio secundario extrajo 2,481 estudios totales publicados de enero de 2008 a noviembre de 2019, de los cuales se seleccionaron 33 estudios primarios (pp. 119-120). En cuanto el análisis del estado del arte, las directrices establecidas por los autores son:

i. La identificación y clasificación de temas relacionados a las pruebas en aplicaciones basadas en la arquitectura de microservicios;
ii. Los enfoques de prueba y herramientas propuestas y usadas en aplicaciones basadas en la aplicaciones basada en la arquitectura de microservicios; y
iii. Los retos asociados a las pruebas de microservicios que han sido reportados en estudios primarios.

Dada la similitud del interés de esta revisión sistemática con respecto al mapeo sistemático, el objetivo de esta revisión es actualizar la información recabada por los autores por aquella disponible actualmente en la literatura blanca. Esto porque, posterior a la publicación del mapeo sistemático, se ha publicado numerosos estudios relacionados a las pruebas en microservicios. A continuación, examinaremos el seguimiento del método, selección de estudios, síntesis de los hallazgos de los estudios y aplicación de herramientas para agilizar la conducción de esta revisión.

# 4 Métodos

Una revisión sistemática de la literatura (RSL) es un estudio secundario de naturaleza empírica fundamentado en la ingeniería de software basada en evidencia (EBSE, por sus traducción al inglés) (Kitchenham et al., 2015, p. 10). Por mencionar que se trata de un estudio, se refiere a que este recaba información de estudios primarios. En esta revisión, se tomó en cuenta la aplicación del método propuesto por Kitchenham et al. (2015), divido en tres fases que guían el desarrollo de una revisión sistemática.

En primera instancia, la planificación de la revisión contempla ($i$) la justificación de la necesidad de una revisión sistemática; ($ii$) la especificación de las preguntas de investigación ($RQ$); ($iii$) el desarrollo del protocolo de la revisión; así como ($iv$) la validación de este. En el protocolo se realizó tanto la planeación tanto de la conducción como del reporte de la revisión. Puntualmente, en la conducción se especificaron los métodos de para la estrategia de búsqueda y la síntesis de datos. Aunque en este trabajo se comentarán algunas actividades del desarrollo de la revisión y los resultados de estas, usted puede consultar con detalle el protocolo (https://rb.gy/pamm52) de esta revisión.

Continuando, la segunda etapa corresponde a la conducción de la revisión, compuesta por la ($i$) la selección de la estrategia de búsqueda; ($ii$) la selección de estudios primarios; ($iii$) la extracción de datos; ($iv$) la selección de la estrategia para la síntesis de datos.

Las primeras dos actividades fueron guiadas por el enfoque sistemático propuesto por Zhang et al (2011) basado en el concepto *quasi-gold standard* (QGS), es decir, el conjunto de estudios recopilados de fuentes de información relacionadas y recabados en un periodo de tiempo específico. Las fuentes de información son conferencias y revistas pertenecientes a un dominio específico, las cuales son reconocidas por la comunidad de una disciplina (p. 628). En otras palabras, en las actividades $i$ y $ii$ se definieron los mecanismos y criterios (inclusión y exclusión) que asistieron la selección de estudios primarios. Más adelante se muestra el procedimiento y los resultados de estas actividades y la siguiente ($iii$).

Por último, la síntesis de datos consiste en el resumen de los hallazgos obtenido a partir del conjunto de estudios primarios seleccionados. Para esta revisión, se optó por el método propuesto por Rodgers et al. (2009) fundado por la síntesis de narrativa. En este trabajo se describen cada una de las fases del método, al igual que los artefactos (herramientas o técnicas) de cada una de ellas. La principal función de la síntesis narrativa es extractar múltiples estudios a través del “uso de palabras y texto para resumir y explicar los hallazgos de la síntesis” (Popay et al., 2006, p. 5).

Finalmente, en la última etapa se reporta la revisión considerando ($i$) la identificación de los interesados en los resultados de la revisión sistemática, ($ii$) la estructura del

reporte; y ($iii$) la validación de este. Como parte de la introducción de este trabajo, se comentó que investigadores y practicante de ingeniería de software representan los beneficiados del desarrollo de la presente revisión. Sobre la estructura del reporte de la revisión se siguió el formato proporcionado por los revisores internos de la Universidad Veracruzana, guiado por la estructura básica de PRISMA recomendada por Kitchenham et al. (2015, p. 354):

a. Título
b. Resumen
c. Introducción
d. Antecedentes
e. Métodos
f. Resultados
g. Discusión
h. Conclusiones
i. Referencias
j. Apéndices

Concluyendo, la validación de este reporte será realizada por revisores internos o externos de la Universidad Veracruzana, con el objetivo de evaluar la calidad de los hallazgos de la presente revisión.

## 4.1 Preguntas de investigación

Dado que las preguntas de investigación ($RQ$) motivan el desarrollo de una revisión (Kitchenham, 2015, p. 32), es importante partir de la discusión en torno a las preguntas (cuatro generales y una específica) del presente trabajo, a modo de justificar su alineación con el objetivo general y trazar la dirección de la investigación. En la Tabla 1 se agrupan las $RQ$ acompañadas de una breve justificación del porqué son de gran valor para el análisis del objeto de estudio.

**Tabla 1**

*Preguntas de investigación de la revisión sistemática de la literatura (RSL)*

| Identificador | Pregunta de investigación | Motivación |
|---|---|---|
| $RQ$-1 | ¿Cuáles son las pruebas de software dedicadas a sistemas basados en la arquitectura de microservicios? | La identificación de las distintas pruebas utilizadas en sistemas basados en la arquitectura de microservicios y su catalogación asistiría a la planeación y diseño de estrategias de prueba. |
| $RQ$-1.1 | ¿En qué contexto se aplicarían las distintas pruebas orientadas a la arquitectura de microservicios? | |

| | | |
|---|---|---|
| $RQ$-2 | ¿Cómo prone la literatura abordar el proceso de prueba orientado a sistemas basados en la arquitectura de microservicios? | La detección de propuestas de la literatura centradas en pruebas dirigidas a la arquitectura de microservicios facilitaría la disposición de información a investigadores y practicantes de ingeniería de software interesados en el desarrollo de microservicios. |
| $RQ$-3 | ¿Cuáles son las herramientas utilizadas para probar sistemas basados en la arquitectura de microservicios? | Ya que se necesita contar con tanta confianza como sea posible durante la etapa de pruebas (Fowler y Lewis, 2014), la implementación de herramientas de software se convierte en un requisito indiscutible. Por ello, es importante identificar tanto las herramientas propuestas por la literatura, como aquellas utilizadas en la industria. |
| $RQ$-4 | ¿Cuáles son los retos existentes asociados al proceso de prueba en la arquitectura de microservicios? | Entendiendo que los métodos tradicionales de prueba enfrentan diferentes dificultades cuando se aplican sobre la arquitectura de microservicios (Gong y Cai, 2023, p. 141). Se precisa analizar el estado actual de las pruebas en microservicios, buscando que futuros trabajos de investigación resuelvan varios de los problemas inherentes al proceso de prueba de microservicios. |

### 4.2 Estrategia de búsqueda

Como se mencionó, esta actividad se empleó el enfoque sistemático propuesto por Zhang et al (2011). Los pasos que guían este método son:

1. Identificación de fuentes de información y motores de búsqueda
2. Establecimiento del QGS (búsqueda manual)
3. Definición o extracción de cadenas de búsqueda
4. Conducción de la búsqueda automatizada
5. Evaluación del rendimiento de la búsqueda

Para la primera actividad, Kitchenham et al. (2015) sugieren que las fuentes de información (*venues*) y motores de búsqueda (*search engines*) más relevantes para la investigación en ingeniería de software son IEEE Digital Library ─ cuyo motor de búsqueda es IEEE Xplore ─ y ACM Digital Library, puesto que ambas cubren la mayor parte de las actas de conferencias y revistas (p. 62). Asimismo, Zhang et al.

(2011) advierten que ScienceDirect y SpringerLink también se consideran fuentes destacadas para el desarrollo de revisiones sistemáticas de ingeniería de software (p. 630). La justificación de la selección de las fuentes y motores de búsqueda para esta revisión se realizará en una sección posterior.

### 4.2.1 Términos de búsqueda

En la Tabla 2 se presentan las palabras clave o *keywords* seleccionadas a partir de las $RQ$ junto a un grupo de términos relacionados por cada keyword. Lo que se busca con este ejercicio es delimitar el dominio del problema de investigación y establecer un conjunto inicial de términos que orienten la búsqueda que se realizará más tarde en el proceso.

**Tabla 2**

*Palabras clave (keyword) relacionada al contexto del problema de investigación de la revisión sistemática de la literatura (RSL)*

| *Keyword* | Término relacionado | Traducción al español |
|---|---|---|
| *Testing* | *Test*, *tests*, *software testing, verification, validation* | Prueba |
| *Microservice* | *Microservices*, *microservices architecture*, *microservices architecture pattern*, *microservice architecture style* | Microservicio |
| *Challenge* | *Problem, difficulty, complexity* | Reto |

### 4.2.2 Cadenas de búsqueda

Una vez se identificaron los términos del dominio, se realizó una búsqueda manual en las fuentes de información elegidas, recabando un conjunto de ocho estudios que al menos responden una $RQ$, mismos que se enlistan en la Tabla 3. Esta búsqueda manual se realizó utilizando distintas cadenas de búsqueda experimentales con el propósito de reducir el impacto del sesgo en la elección de una cadena de búsqueda para la selección automatizada de estudios primarios.

**Tabla 3**

*Estudios primarios relevantes seleccionados manualmente para la revisión sistemática de la literatura (RSL)*

| Identificador | Referencia |
|---|---|
| $P$-1 | Camilli, M., Guerriero, A., Janes, A., Russo, B. y Russo, S. (2022). Microservices integrated performance and reliability testing. *Proceedings of the 3rd ACM/IEEE International* |

| | |
|---|---|
| | *Conference on Automation of Software Test (AST '22)*, Nueva York, NY, EUA, 29-39. https://doi.org/10.1145/3524481.3527233 |
| $P$-2 | Ding, H., Cheng, L. y Li, Q. (2020). An Automatic Test Data Generation Method for Microservice Application. *2020 International Conference on Computer Engineering and Application (ICCEA)*, Guangzhou, China, 188-191. https://doi.org/10.1109/ICCEA50009.2020.00048 |
| $P$-3 | Duan, T., Li, D., Xuan, J., Du, F., Li, J., Du, J. y Wu, S. (2021). Design and Implementation of Intelligent Automated Testing of Microservice Application. *2021 IEEE 5th Information Technology, Networking, Electronic and Automation Control Conference (ITNEC)*, Xi'an, China, 1306-1309. https://doi.org/10.1109/ITNEC52019.2021.9587260 |
| $P$-4 | Elsner, D., Bertagnolli, D., Pretschner, A. y Klaus, R. (2022). Challenges in regression test selection for end-to-end testing of microservice-based software systems. *Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test (AST '22)*, Nueva York, NY, EUA, 1-5. https://doi.org/10.1145/3524481.3527217 |
| $P$-5 | Gong, J. y Cai, L. (2023). Analysis for Microservice Architecture Application Quality Model and Testing Method. *2023 26th ACIS International Winter Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD-Winter)*, Taiyuan, Taiwan, 141-145. https://doi.org/10.1109/SNPD-Winter57765.2023.10223960 |
| $P$-6 | Li, H., Wang, J., Dai, H. y Lv, B. (2020). Research on Microservice Application Testing System. *2020 IEEE 3rd International Conference on Information Systems and Computer Aided Education (ICISCAE)*, Dalian, China, 363-368. https://doi.org/10.1109/ICISCAE51034.2020.9236829 |
| $P$-7 | Meng, X, Duan, X., Tao, W., Luan, Y., Zhang, J. y Wu, D. (2021). Modeling and verification of Industrial Microservice Architecture Based on Formal Methods. *2021 China Automation Congress (CAC)*, Beijing, China, 3776-3780. https://doi.org/10.1109/CAC53003.2021.9727653 |
| $P$-8 | Wu, C. F., Ma, S. P., Shau, A. C. y Yeh, H. W. (2022). Testing for Event-Driven Microservices Based on Consumer-Driven Contracts and State Models. *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*, Japan, 467-471. https://doi.org/10.1109/APSEC57359.2022.00064 |

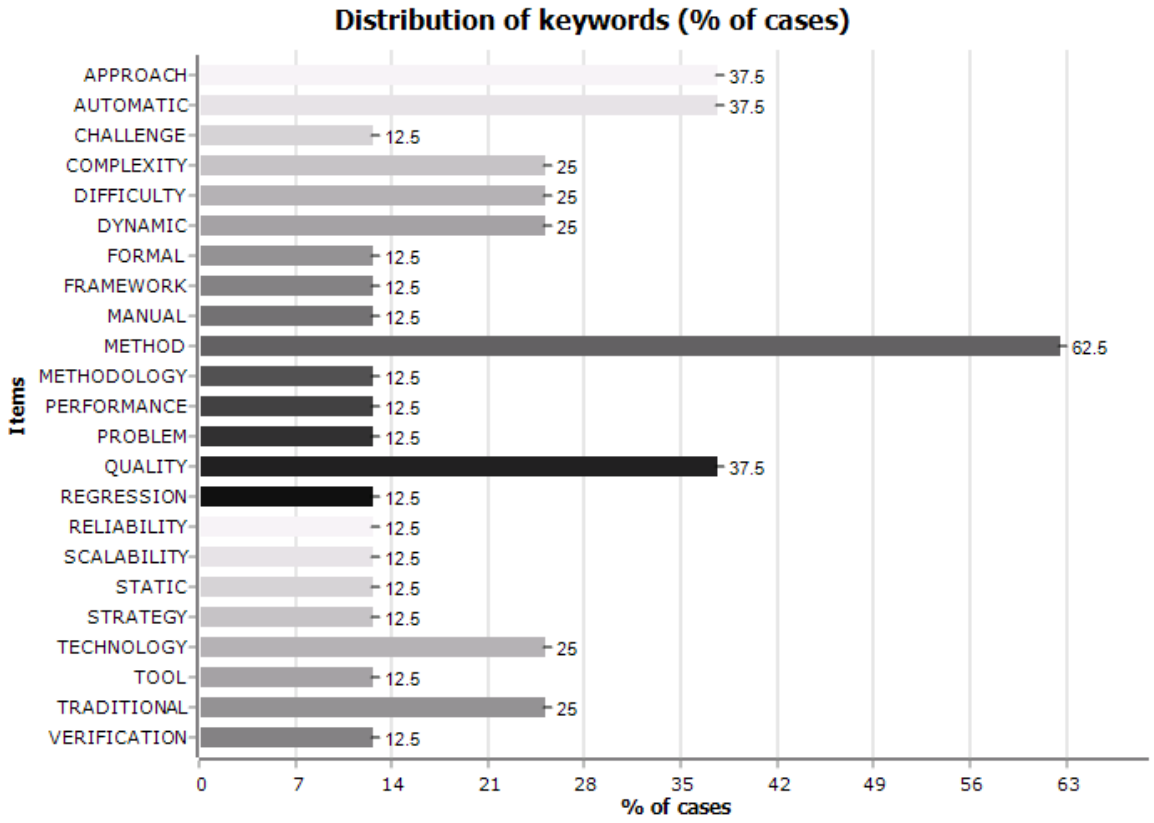
Después de la selección manual de estudios, el conjunto consecuente se sometió a un análisis para la extracción de *keywords* por medio de la herramienta WordStat, en su versión 2023.2. Esto con el objetivo de realizar una extracción objetiva de términos de búsqueda, como se recomienda en el trabajo de Zhang et al. (2011). La

Figura 1 muestra el resultado del análisis de la distribución de las palabras clave identificadas en el título, *abstract* o *keywords* del conjunto de estudios relevantes seleccionados (p. 629).

La identificación de *keywords* dio paso al diseño de cuatro cadenas de búsqueda (Tabla 5) para la extracción de estudios primarios. Además, se optó por utilizar la cadena presentada en el mapeo sistemático de Waseem et al. (2020) con la finalidad de evaluar el esfuerzo de investigación (número de trabajos de investigación arrojados por la cadena) efectuado posterior a la publicación de su trabajo, cuyo alcance comprende estudios publicados desde enero de 2008 a noviembre de 2019 (p. 120). Cada cadena fue probada mediante el motor de búsqueda IEEE Xplore para valorar su rendimiento a través de la sensibilidad y precisión, empleando las métricas *sensitivity* ($\frac{R_{retrieved}}{T}$) y *precision* ($\frac{R_{retrieved}}{N}$) (Zhang et al., 2011, p. 627), respectivamente; donde $R_{retrieved}$ representa el número de estudios relevantes recuperados por una cadena de búsqueda, $T$ representa el total de estudios relevantes seleccionados manualmente y $N$ representa el número total de estudios recuperados por una cadena de búsqueda.

**Figura 1**

*Distribución de las palabras clave (keywords) en los estudios primarios relevantes seleccionados manualmente para la revisión sistemática de la literatura (RSL)*




*Nota*. Análisis devuelto por la herramienta WordStat en su versión 2023.2.

Además de las métricas señaladas, Zhang et al. (2011) definen una clasificación de estrategias de búsqueda en función de la sensibilidad obtenida por la cadena. Los autores señalan que aquellas cadenas cuyo valor de *recall* sea $\geq 0.8$, son candidatas para proceder con el proceso de selección (p. 630). La Tabla 4 muestra la clasificación de estrategias de búsqueda para una cadena.

**Tabla 4**

*Escalas de estrategias de búsqueda*[a]

| Estrategia | *Sensitivity* (%) | *Precision* (%) | Observaciones |
|---|---|---|---|
| Sensibilidad alta | 85-90 | 7-15 | Máxima sensibilidad a pesar de una precisión baja; factible para el desarrollo de una revisión sistemática. |
| Precisión alta | 45-58 | 25-60 | Máxima tasa de precisión pese a una baja sensibilidad. |
| Óptima | 80-99 | 20-25 | Maximiza tanto la sensibilidad como la precisión; factible para el desarrollo de una revisión sistemática. |
| Aceptable | 72-80 | 15-25 | Sensibilidad y precisión razonables; factible para el desarrollo de una revisión sistemática, si y solo si, el valor de la sensibilidad es igual al límite superior (80). |

*Nota*. [a]Zhang et al. (2011, p. 630).

**Tabla 5**

*Cadenas de búsqueda candidatas para la selección automatizada de estudios primarios*

| Identificador | Cadena de búsqueda | $N$ | $R_{retrieved}$ | *Sensitivity* | *Precision* |
|---|---|---|---|---|---|
| $S$-1 | ((microservice* OR micro service* OR micro-service* OR microservices architect* OR microservices design) AND test*) | 1231 | 8 | $^{8}/_{8} = 1$ | $^{8}/_{1231} = 0.006$ |
| $S$-2 | ((microservice* OR micro service* OR micro-service* OR | 1332 | 8 | $^{8}/_{8} = 1$ | $^{8}/_{1332} = 0.006$ |

| | | | | | |
|---|---|---|---|---|---|
| | "microservice? architect*") AND (test* OR verification)) | | | | |
| $S$-3 | ((microservice* OR "microservice? architect*") AND (test* OR verification OR validation)) | 580 | 8 | $^{8}/_{8} = 1$ | $^{8}/_{580} = 0.013$ |
| $S$-4 | (microservice AND architect* AND test*) | 396 | 7 | $^{7}/_{8} = 0.875$ | $^{7}/_{396} = 0.017$ |
| $S$-5 | ((microservice* OR micro service* OR micro-service* OR "microservice? architect*") AND (test* OR verification) AND (tool OR system OR method OR strateg* OR model OR practice OR pattern OR approach OR algorithm) AND (challenge OR complexit* OR difficult* OR problem)) | 466 | 5 | $^{5}/_{8} = 0.625$ | $^{5}/_{466} = 0.010$ |

Con base en la evaluación de las cinco cadenas propuestas, se aprecia que la cadena $S$-3 cuenta con el rendimiento más favorable para proceder con el desarrollo de la revisión. Aun cuando la cadena $S$-4 arroja una precisión mayor (*precision*), $S$-3 se ve favorecida por su sensibilidad (*sensitivity*), pues es mayor a la de $S$-4. A partir de los resultado anteriores, fue posible continuar con los pasos subsecuentes sugeridos por Zhang et al. (2011).

### 4.2.3 Selección de fuentes

La selección de estudios primarios se realizó mediante las fuentes de información (*venues*) y motores de búsqueda (*search engines*) recomendados tanto por Kitchenham et al. (2015, p. 62) como por Zhang et al. (2011, p. 630), como se aprecia en la Tabla 6.

**Tabla 6**

*Fuentes de información (venues) y motores de búsqueda (search engines) elegidos para la selección automatizada de estudios primarios*

| Fuente de información | Motor de búsqueda | URL |
|---|---|---|
| IEEE Digital Library | IEEE Xplore | https://ieeexplore.ieee.org/ |
| ACM Digital Library | ACM Digital Library | https://dl.acm.org/ |
| Springer | SpringerLink | https://link.springer.com/ |
| Elsevier | ScienceDirect | https://www.sciencedirect.com/ |

## 4.3 Selección de estudios primarios

En esta actividad se establecieron los criterios de inclusión y exclusión, los cuales ayudaron a acotar el conjunto de estudios obtenidos por la cadena búsqueda seleccionada.

### 4.3.1 Criterios de selección de estudios primarios

Este conjunto de criterios ayudó a la detección de los estudios se considerados relevantes para el desarrollo de esta revisión. La Tabla 7 concentra los criterios de inclusión aplicados ($C_I$), mientras que la Tabla 8 muestra los criterios de exclusión ($C_E$).

**Tabla 7**

*Criterios de inclusión para la selección de estudios primarios*

| Identificador | Criterio |
|---|---|
| $C_I$-1 | Se recabarán y analizarán exclusivamente estudios escritos en el idioma inglés. |
| $C_I$-2 | Se considerarán estudios cuya temporalidad se posicione en el periodo comprendido de enero de 2018 a noviembre de 2023. |
| $C_I$-3 | El título o palabra clave (*keyword*) especificada por los autores del estudio incluyen algún término relacionado a las pruebas orientadas a la arquitectura de microservicios. |
| $C_I$-4 | El resumen (*abstract*) del estudio presenta indicios de responder al menos una pregunta de investigación. |
| $C_I$-5 | Tras una lectura completa del estudio, se valida que el estudio responde alguna pregunta de investigación ($RQ$). |

***Tabla 8***

*Criterios de exclusión para la selección de estudios primarios*

| Identificador | Criterio |
|---|---|
| $C_E$-1 | Se descartarán estudios a los que no se tenga acceso por medio del convenio entre el Consorcio Nacional de Recursos de Información Científica y Tecnológica (CONRICyT) y la Universidad Veracruzana. |
| $C_E$-2 | Se descartarán estudios distinto a libros, artículos publicados o preaprobados, al igual que actas de conferencias y revistas. |
| $C_E$-3 | Se descartarán estudios duplicados. |

### 4.3.2 Procedimiento de selección de estudios primarios

En la búsqueda de una aplicación rigurosa de los criterios de selección predefinidos, es necesario seguir un proceso sistemático que permita agilizar la recopilación de estudios primarios mediante la cadena seleccionada. Los pasos presentados a continuación componen el flujo de trabajo que aplicado durante la selección de estudios primarios:

i. Filtrado del número total de estudios recuperados por la cadena de búsqueda ($N$) a través del motor de búsqueda, aplicación de los criterios $C_I$-1, $C_I$-2 y $C_E$-1.

ii. Revisión del estudio, aplicación de los criterios $C_E$-2 y $C_E$-3.

iii. Lectura rápida del estudio, aplicación de los criterios $C_I$-3 y $C_I$-4.

iv. Lectura completa del estudio, aplicación del criterio $C_E$-5.

## 4.4 Extracción de datos

En esta actividad se establecieron los datos que fueron extraídos de los estudios primarios, así como los mecanismos por lo que se recabó esta información. Kitchenham et al. (2015) señalan que los detalles de publicación y la información necesaria para responder las $RQ$ son los datos imprescindibles que deben sustraerse (p. 51). La Tabla 9 muestra los datos que se extraerán por cada estudio primario que integrará esta revisión.

**Tabla 9**

*Plantilla para la extracción de datos de los estudios primarios*

| Dato | Descripción |
|---|---|
| Identificador | Identificador único cuya nomenclatura es $P$-$X$; donde $P$ representa un estudio primario y $X$ el número secuencial asignado. |
| Título | Título del estudio primario. |
| Autores | Nombre y apellido del autor del estudio primario. |
| Año de publicación | En correspondencia con $C_I$-2. |
| Tipo de publicación | En correspondencia con $C_E$-2. |
| *Venue* | Nombre de la fuente de información que publicó el estudio primario. |
| DOI | Identificador de objeto digital (DOI, por sus siglas en inglés) asignado al estudio primario. |

| | |
|---|---|
| *Keyword* | Palabra clave especificada por el estudio primario relacionada con el enfoque de la revisión; en correspondencia con $C_I$-3. |
| *Abstract* | Resumen del estudio primario; en correspondencia con $C_I$-4. |
| Preguntas de investigación relacionadas | Pregunta de investigación ($RQ$) que motiva esta revisión. |
| Tipo de prueba | En correspondencia con $RQ$-1. |
| Nivel de prueba | En correspondencia con $RQ$-1. |
| Etapa del ciclo de vida de desarrollo de software en la que se aplica (SDLC, por sus siglas en inglés) | En correspondencia con $RQ$-1.1. |
| Propuesta de la literatura blanca | En correspondencia con $RQ$-2. |
| Herramienta | En correspondencia con $RQ$-3. |
| Retos | En correspondencia con $RQ$-4. |

## 4.5 Estrategia para la síntesis de datos

La estrategia que utilizado en esta revisión es la síntesis narrativa, cuyo fundamento se encuentra en el trabajo de Rodgers et al. (2009). En este, se proponen cuatro actividades y un grupo de herramientas o técnicas por cada una.

### 4.5.1 Desarrollo de una teoría

En esta actividad inicial se contextualizan y formulan un conjunto de preguntas ($RQ$) que guían la investigación en torno al objeto de estudio. Por ello se considera que esta etapa es crucial, puesto que dirige el proceso de selección de estudios primarios, su síntesis y consecuente análisis.

### 4.5.2 Desarrollo de una síntesis preliminar

Rodgers et al. (2009) proponen algunas herramientas y técnicas para la síntesis preliminar de los estudios, las cuales fueron seleccionadas dada la naturaleza cuantitativa de su caso de estudio (revisión sistemática). Esto es de consideración, pues como señalan Popay et al. (2006), la manera (herramientas y técnicas) en que el autor debe abordar la síntesis preliminar depende del tipo de investigación que esté desarrollando, ya sea cualitativa, cuantitativa o ambas (p. 7). En este caso, se seleccionó el siguiente par de técnicas:

1. Tabulación de datos
2. Descripción textual

La primera, consiste en la extracción de información de cada estudio primario para agruparla en una tabla; los autores mencionan que a pesar de ser prematuro evaluar la calidad del estudio, es una buena oportunidad para realizar una conjetura inicial con respecto a este. Mientras que la segunda requiere realizar un resumen breve

de estudio, es una técnica complementaria a la tabulación de datos. Se recomienda utilizarla en la siguiente actividad (exploración de las relaciones dentro y entre estudios), sin embargo, es válido aplicarla desde este punto (Rodgers et al., 2009, pp. 53-54).

#### 4.5.3 Exploración de las relaciones dentro y entre estudios

Para esta actividad se detectan dos tipos de relaciones, para la primera se analizan las características de un estudio y sus hallazgos; para la segunda, se comparan estos hallazgos en relación con otros estudios (Rodgers et al., 2009, p. 58). La técnica seleccionada fue seleccionada:

1. Desarrollo de modelos conceptuales/red de ideas/mapeo conceptual

Esta técnica brinda un apoyo visual que muestra las relaciones de los estudios primarios; este esquema estructura la narración de la síntesis con un mayor pragmatismo (Rodgers et al., 2009, pp. 62-64).

#### 4.5.4 Evaluación de la robustez del producto de la síntesis

Finalmente, el proceso de síntesis culmina con la evaluación general sobre la calidad de la evidencia disponible, obtenida de los estudios primarios, para la realización de inferencias y conclusiones. La técnica empleada durante esta actividad fue:

1. Reflexión crítica sobre el proceso de síntesis

Consiste en el análisis del proceso de la síntesis, por lo que se pueden discutir limitaciones o problemas surgidos durante esta etapa de la revisión (Rodgers et al., 2009, p. 67).

### 4.6 Conducción de la revisión

Una vez se definido el procedimiento de selección de estudios primarios en el protocolo de la revisión, se procedió a aplicar cada una de las actividades. A lo largo de este apartado se identifican herramientas que asistieron la conducción de la revisión, con la finalidad de reducir la duración de esta etapa. La Tabla 10, 11, 12 y 13 muestran la actualización de los estudios primarios seleccionados por cada una de las etapas del procedimiento de selección de estudios descrito secciones atrás. Estas tablas se acompañan de una gráfica de modo que se visualicen los resultados de cada etapa con mayor facilidad (Figura 2, 3, 4 y 5). Asimismo, en el Tabla A1 del trabajo se incluyen la variantes de la cadena de búsqueda ($S$-3) seleccionada aplicadas en cada uno de los motores de búsqueda elegidos.

### 4.6.1 Herramientas

Pese a que este apartado pudo incluirse en la planeación de la revisión, realmente no fue hasta este punto que se utilizaron herramientas. Cada una de las herramientas enlistadas cumplió una función específica para la selección de estudios:

- Parsifal
- Zotero
- MAXQDA

Parsifal es un herramienta web desarrollada para la gestión de revisiones sistemáticas. En esta revisión se trabajó con la versión 2.2, utilizada para la importación de estudios mediante un archivo BibText, los cuales se pueden exportar en los motores de búsqueda IEEE Xplore, ACM Digital Library y ScienceDirect. Esto permitió clasificar los estudios en las categorías: aceptado, rechazado, desclasificado y duplicado. Incluso ofrece la capacidad de añadir criterios de inclusión y exclusión para catalogar los estudios, permitiendo contar con un mayor control durante esta actividad. En el caso de SpringerLink, el motor permite únicamente permite exportar un archivo con extensión .xls, por lo cual se seleccionaron y se agregaron manualmente los estudios a Parsifal (https://rb.gy/0qufvp).

Una vez terminada la selección de estudios, se empleó la herramienta Zotero en su versión 6.0.30. Desafortunadamente Parsifal solo cuenta con la función de exportación de un archivo con extensión .xls, por lo que se agregaron manualmente los estudios a Zotero (https://rb.gy/hwcyy6). Esta herramienta facilita la gestión de documentos y ofrece distintas opciones de exportación, entre las que se encuentra la opción de exportar colecciones en formato RIS. Mismas que pueden ser importadas en MAXQDA.

Finalmente, se utilizó la versión 22.0.8 de MAXQDA junto a la licencia Analytics Pro 2022 Teaching. La herramienta cuenta, entre tantas funciones, con la capacidad de extraer y sintetizar datos. Particularmente, MAXQDA facilita implementación de un sistema de códigos personalizable para el análisis de estudios por medio de un marcado o subrayado de fragmentos de los estudios. Además, por cada uno de los códigos creados posibilita la exportación de un archivo .xlsx que contiene una tabla con la información extraída. Esto fue sumamente valioso ya que se diseñó un sistema de códigos (https://rb.gy/5qs3iy) en el que se incluyó cada una de las preguntas de investigación ($RQ$). De este modo, agilizando la aplicación de las técnicas del desarrollo de la síntesis preliminar de la síntesis narrativa propuesta por Rodgers et al. (2009).

**Tabla 10**

*Estudios seleccionados posterior al filtrado del número total de estudios recuperados por la cadena de búsqueda a través de cada motor de búsqueda*

| Motor de búsqueda | $N$ |
|---|---|
| IEEE Xplore | 508 |
| ACM Digital Library | 84 |
| SpringerLink | 517 |
| ScienceDirect | 16 |

*Nota.* Se aplicaron los criterios de selección $C_I$-1, $C_I$-2 y $C_E$-1.

**Figura 2**

*Estudios seleccionados posterior al filtrado del número total de estudios recuperados por la cadena de búsqueda a través de cada motor de búsqueda*

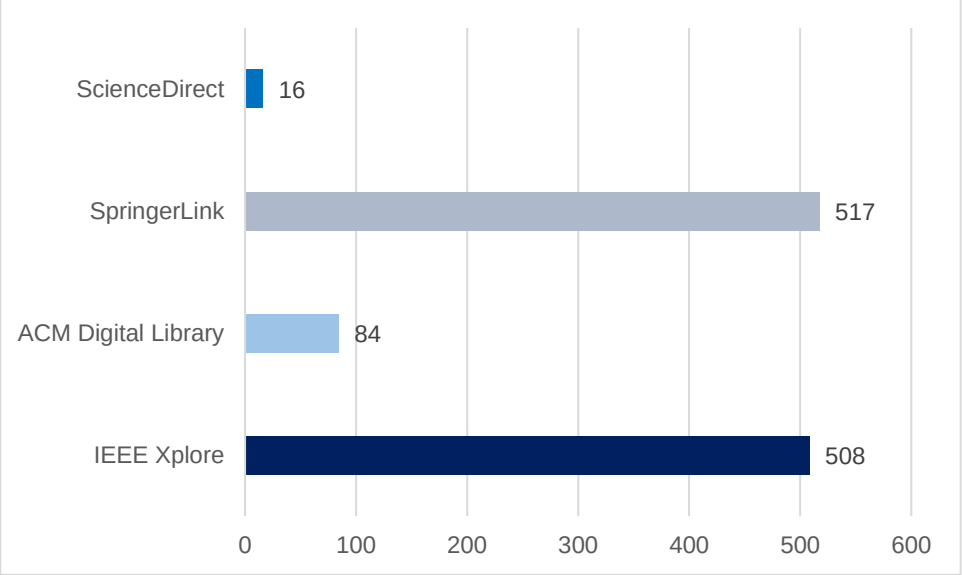


**Tabla 11**

*Estudios seleccionados posterior a la revisión del estudio*

| Motor de búsqueda | $N$ |
|---|---|
| IEEE Xplore | 502 |
| ACM Digital Library | 84 |
| SpringerLink | 515 |
| ScienceDirect | 16 |

*Nota.* Se aplicaron los criterios de selección $C_E$-2 y $C_E$-3.

**Figura 3**

*Estudios seleccionados posterior a la revisión del estudio*

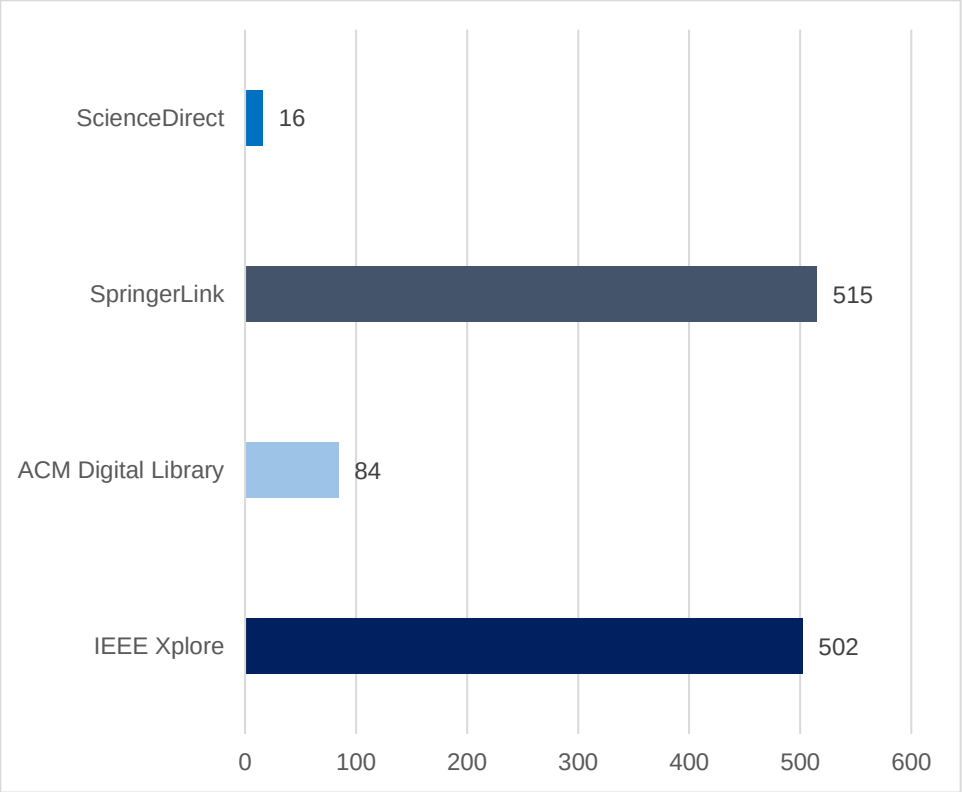


**Tabla 12**

*Estudios seleccionados posterior a la lectura rápida del estudio*

| Motor de búsqueda | $N$ |
|---|---|
| IEEE Xplore | 50 |
| ACM Digital Library | 1 |
| SpringerLink | 2 |
| ScienceDirect | 0 |

*Nota.* Se aplicaron los criterios de selección $C_I$-3 y $C_I$-4.

**Figura 4**

*Estudios seleccionados posterior a la lectura rápida del estudio*

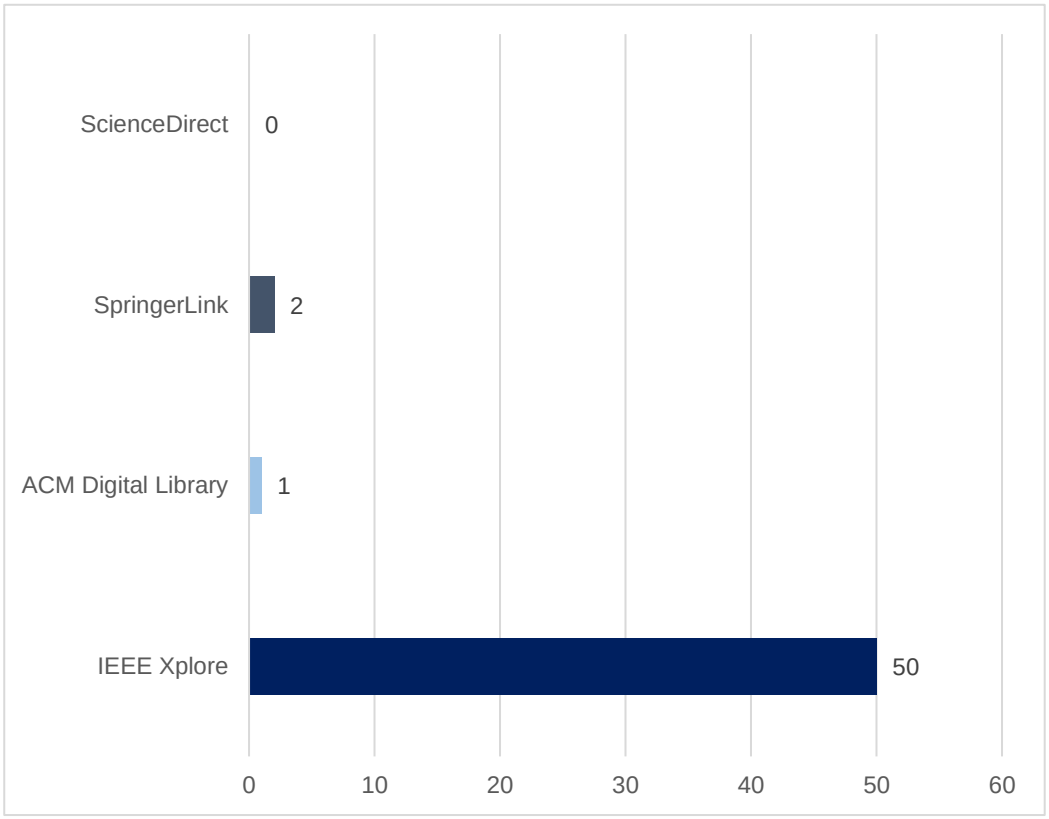


**Tabla 13**

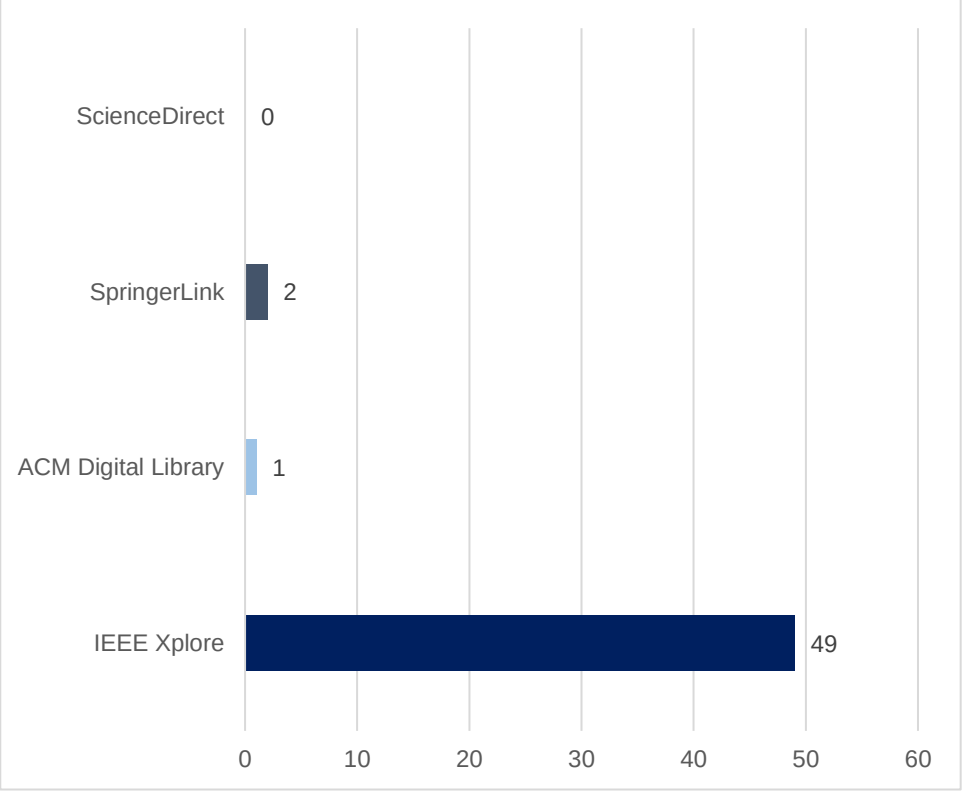
*Estudios seleccionados posterior a la lectura completa del estudio*

| Motor de búsqueda | $N$ |
|---|---|
| IEEE Xplore | 49 |
| ACM Digital Library | 1 |
| SpringerLink | 2 |
| ScienceDirect | 0 |

*Nota.* Se aplicó el criterio de selección $C_I$-5.

**Figura 5**

*Estudios seleccionados posterior a la lectura completa del estudio*



## 5 Resultados

La conducción de la revisión dio como resultado la selección de 52 estudios primarios, de los cuales 47 corresponden a ~~actas de~~ conferencias y el resto (5) a artículos de revistas. En la Figura 6 se puede apreciar la distribución de los estudios seleccionados. Del mismo modo, en la Figura 7 se puede observar la distribución de estudios por año de publicación.

**Figura 6**

*Distribución de la selección de estudios primarios por tipo*

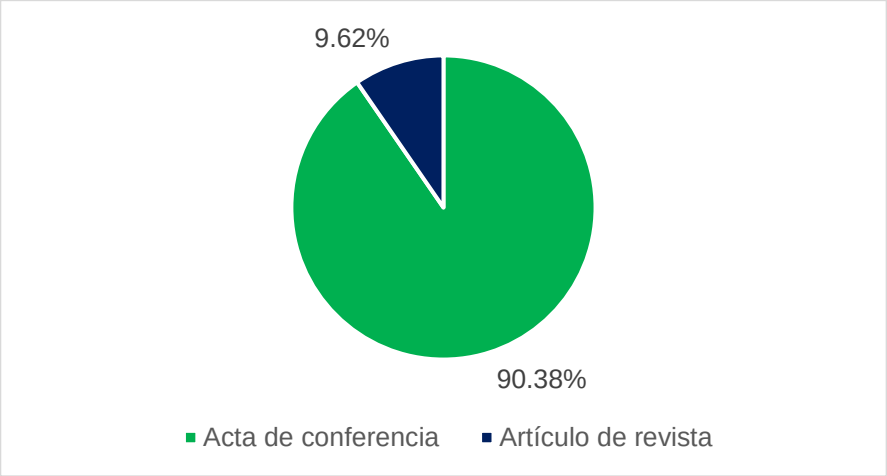
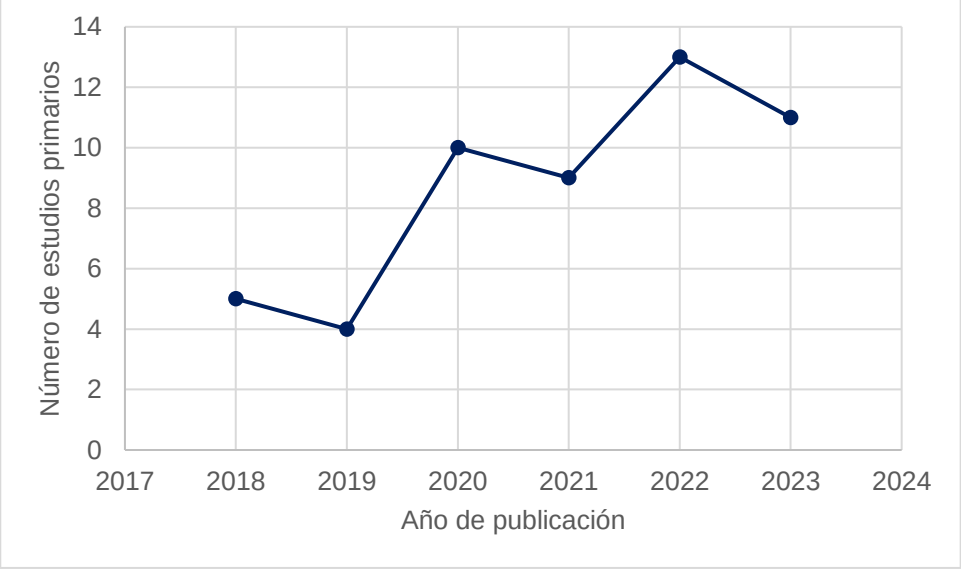



**Figura 7**

*Distribución de la selección de estudios primarios por año de publicación*



Como se puede observar, existe una tendencia creciente en cuanto a la investigación sobre las pruebas de software en microservicios o temas afines a esta actividad. Retomando lo comentado en los primeros apartados de este trabajo, es justificable el desarrollo de una revisión sistemática que agrupe información relacionada a las pruebas en microservicios dada la perceptible preocupación de la academia por resolver varios de los problemas asociados a este estilo arquitectónico.

El propósito de este apartado es comentar los principales hallazgos de la investigación clasificados a partir de cada pregunta de investigación ($RQ$). Asimismo, se presenta una categorización de los estudios primarios según su contribución a la respuesta de cualquiera de las $RQ$. Para consultar la referencia de alguno de los estudios usted podrá dirigirse al apartado de Referencias.

## 5.1 Pruebas orientadas a sistemas basados en la arquitectura de microservicios

### 5.1.1 Enfoque de prueba

El objetivo principal de las pruebas de software es verificar el sistema con el comportamiento esperado [8]. Las técnicas de prueba de software generalmente se clasifican en las siguientes categorías: caja negra, caja blanca y caja gris [9] (Sotomayor et al., 2019, p 1). Por ejemplo, las pruebas punto-a-punto son una técnica de prueba de caja negra funcional; mientras que las pruebas de pruebas de rendimiento y estrés son pruebas no funcionales (Sotomayor et al., 2019, p. 2).

Las pruebas de caja gris de arquitecturas de microservicios permiten a los evaluadores obtener información sobre el comportamiento de los microservicios internos, en términos de cobertura y fallas (Giamattei et al., 2022, p. 10). No obstante, las pruebas de caja negra generalmente se considera el enfoque más adecuado para la arquitectura de microservicios (Giamattei et al., 2022, p. 1)

#### 5.1.1.1 Pruebas funcionales

Las pruebas funcionales garantizan que varias características del sistema funcionen de acuerdo con las especificaciones o expectativas de la funcionalidad del sistema. Los casos de prueba generalmente se basan en las especificaciones de los componentes de software y cada prueba corresponde a un requisito o característica de software determinado. Esto ayuda a garantizar que las partes del paquete de software general funcionen de forma independiente. Centrarse en los requisitos del software garantiza que el resultado sea coherente con las expectativas del usuario final. Para simular la interacción del usuario con sistemas de microservicios, consideramos el sistema como una caja negra y utilizamos sus interfaces de usuario sin conocer los detalles internos (Smith et al., 2023, p. 1).

Además de las pruebas funcionales, es vital evaluar que el sistema responde adecuadamente a diversas cantidades de usuarios y solicitudes en carga simultánea. Estas pruebas de carga pueden acompañar a las pruebas funcionales

mientras se monitorean los tiempos de respuesta del sistema (Smith et al., 2023, p. 1).
Las pruebas de funcionalidad consideran el sistema como una caja negra y no son conscientes de la descomposición interna en microservicios, que a menudo refleja la visión del usuario. El punto de referencia de prueba de interfaz de usuario (basado en web) ayuda a garantizar la coherencia durante la ejecución de las acciones del usuario. La prueba de funcionalidad a través de la interfaz de usuario garantiza que el sistema muestre la información de manera correcta y consistente. Si hay variabilidad en la información mostrada, podría afectar en gran medida la experiencia del usuario (Smith et al., 2023, p. 3).

### 5.1.2 Característica o atributo de calidad

La mayor parte de las características de calidad de arquitecturas de microservicios pueden medirse a través de las pruebas punto-a-punto. Más adelante se detalla sobre este tipo de pruebas. A continuación, se presentan algunas pruebas y métricas significativas para los microservicios.

#### 5.1.2.1 Pruebas de rendimiento

El rendimiento se prueba a varios niveles (por ejemplo, a nivel de unidad [47] y de integración [30]). De ahí la importancia de disponer de la aplicación completa (incluidos todos sus componentes) para probar a fondo su rendimiento (Eismann et al., 2020, p. 3). En muchos casos, una prueba de rendimiento más larga proporciona más información sobre una aplicación. Como resultado, una prueba de rendimiento puede tardar horas o incluso días en completarse (Eismann et al., 2020, p. 3).

La prueba de carga es el proceso de simular la demanda del sistema que prueba el comportamiento en diversas condiciones. El término "carga" en este caso se refiere a la tasa o número de usuarios y solicitudes que acceden a un sistema determinado [3]. Las pruebas de carga ayudan a garantizar que las acciones del usuario en el sistema sean estables al evaluar cómo reacciona el sistema a diversas cantidades de carga.

Hay tres formas principales en que se pueden ejecutar las pruebas de carga. Estas técnicas de generación de carga incluyen el uso de usuarios reales para generar la carga, el uso de controladores de carga o la implementación de pruebas de carga en plataformas especiales [4]. Evaluar el comportamiento de un sistema en múltiples cantidades de carga puede resaltar áreas de mejora, como cuellos de botella o áreas de falla dentro del sistema. Estos problemas a menudo solo se pueden descubrir mediante pruebas de carga porque solo ocurren o son visibles bajo una cierta cantidad de carga en el sistema (Smith et al., 2023, p. 2).

La evaluación del rendimiento es parte integral del ciclo de desarrollo de software. Un enfoque habitual para evaluar el rendimiento son las pruebas de carga [24]: un controlador de carga genera peticiones sintéticas al sistema sometido a prueba de acuerdo con un modelo de carga de trabajo previamente definido. La

representatividad es un requisito común de las pruebas de carga, es decir, que las características de la carga de trabajo generada -por ejemplo, las tasas de llegada y los parámetros de las peticiones- se correspondan con las características de la carga de trabajo de producción [11] (Schulz et al., 2019, p. 1). Apoyando esto, Eismann et al. (2020) comentan que una prueba de rendimiento debe ejecutarse bajo una carga de trabajo que sea realista y representativa del uso real de la aplicación. Por ello, idealmente el entorno de prueba debería ser igual al entorno de producción (Eismann et al., 2020, p. 3).

Para prepararse para la carga de trabajo de producción prevista, una prueba de carga simula usuarios reales enviando solicitudes a los puntos finales (por ejemplo, puntos finales REST) (Schulz et al., 2019, p. 2).

Las pruebas de carga tienen como objetivo identificar posibles cuello de botella y lentitud. Esto es importante porque estos problemas pueden afectar significativamente la experiencia del usuario. Sin embargo, con las pruebas de carga, podríamos considerar que el sistema es una caja gris y centrarnos en los puntos finales expuestos por el *middleware*. Como se mencionó anteriormente, las pruebas de carga de los sistemas de microservicios son vitales para garantizar que los distintos microservicios reaccionen bien a diferentes cantidades de carga. Esto puede ayudar a identificar áreas de cuellos de botella y fallas dentro del sistema (Smith et al., 2023, p. 3).

Además de las pruebas de carga, las pruebas de estrés tienen como objetivo descubrir que el servicio de arquitectura de microservicios puede manejar solicitudes grandes (Maulana y Raharja, 2022, p. 5).

Por otro lado, Sedghpour et al. (2023) definen un conjunto de métricas significativas para la evaluación del rendimiento de un sistema (p. 4):

- Capacidad: rendimiento máximo admitido, es decir, el número de solicitudes exitosas por segundo para las cuales el tiempo de respuesta está por debajo de un umbral de 100 ms.
- Latencia: percentil 95 del tiempo de respuesta asociado a las solicitudes. Este percentil se utiliza porque se correlaciona bien con la experiencia del usuario [28] y, al mismo tiempo, proporciona una señal menos ruidosa que los percentiles más altos.
- Tiempo de CPU por solicitud: tiempo necesario para que la CPU procese cada solicitud.
- Porcentaje de uso de CPU: cantidad de CPU utilizada por una instancia de microservicio en relación con la cantidad de CPU asignada.

Algo similar sugieren Jin et al. (2020), pues ellos comentan que, para microservicios, el tiempo de respuesta y latencia son indicadores clave de rendimiento (Jin et al., 2020, p. 6).

Por último, un ejemplo de las pruebas de rendimiento son las pruebas de rendimiento de un solo usuario. El propósito de dichas pruebas es comprender el

flujo de control de las solicitudes de los usuarios e identificar segmentos que consumen la mayor parte del tiempo de respuesta (Camilli et al., 2022, p. 2).

#### 5.1.2.2 Pruebas de fiabilidad

Gong y Cai (2023) señalan que la medición de la disponibilidad y tolerancia a fallos de los microservicios se da por técnicas como la simulación de defectos (p. 4). Más adelante se comentan propuestas de la literatura relacionadas a esto.

En el caso de la tolerancia a fallos de aplicaciones de microservicios, esta puede probarse juzgando los mecanismos de fusión de servicios, limitación de flujo, degradación, equilibrio de carga, etc. Por otra parte, la disponibilidad se puede medir por la cantidad de instancias de servicios clave además del porcentaje de tiempo en el estado disponible (Gong y Cai, 2023, p. 3).

#### 5.1.2.3 Pruebas de mantenibilidad

La trazabilidad se puede medir en función de si la aplicación de arquitectura de microservicio implementa una característica de trazabilidad del enlace de servicio o mediante la proporción de servicios rastreables (Gong y Cai, 2023, p. 3).

#### 5.1.2.4 Pruebas de escalabilidad

Las pruebas de escalabilidad verifican la capacidad del sistema de destino para cumplir con los requisitos de rendimiento en situaciones de carga exigentes (Camilli et al., 2022, p. 2). La escalabilidad se puede medir por el beneficio del crecimiento de su capacidad en relación con el costo requerido para escalar el sistema (Gong y Cai, 2023, p. 3).

#### 5.1.2.5 Pruebas de compatibilidad

La interoperabilidad de las aplicaciones de arquitectura de microservicios se puede medir en función de si las interfaces externas de cada servicio pueden seguir una buena definición de especificación de interfaz. Además, la coexistencia se puede medir por la aparición de conflictos, dependencias circulares, etc. (Gong y Cai, 2023, p. 3).

#### 5.1.2.6 Pruebas de seguridad

Las pruebas de seguridad de software son parte del dominio de pruebas de software que se centra en las propiedades de seguridad de las aplicaciones de software. Se divide en pruebas de seguridad funcional y pruebas de seguridad de vulnerabilidad [5]. Las pruebas de seguridad funcional garantizan que las funciones de seguridad se implementen correctamente en una aplicación. Las pruebas de seguridad de vulnerabilidades se centran en revelar riesgos de seguridad y vulnerabilidades que pueden hacer que un sistema de software sea explotable por atacantes. Las pruebas de seguridad de vulnerabilidades pueden ser estáticas o dinámicas. Las pruebas estáticas se centran en identificar configuraciones erróneas y vulnerabilidades en el código fuente. Mientras que las pruebas dinámicas se centran

en detectar vulnerabilidades y debilidades de seguridad en las aplicaciones evaluándolas durante su ejecución (Ünver and Britto, 2023, p. 1).

### 5.1.3 Niveles de prueba

En el estilo arquitectónico de microservicios, cada estrategia de pruebas tiene como objetivo proporcionar confianza de corrección en cada microservicio, o en un conjunto de ellos. Como se detalla en [3], las pruebas unitarias tienen la responsabilidad de validar el comportamiento interno de un microservicio. Las pruebas de integración tienen la responsabilidad de validar las vías de comunicación y las interacciones entre los microservicios compuestos; su objetivo es comprobar que cada microservicio puede comunicarse con los demás. Como tales, a menudo se consideran como una retroalimentación rápida hacia la integración. Del mismo modo, las pruebas de contrato tienen la responsabilidad de comprobar las interacciones en el límite de la aplicación, asegurando que cada microservicio de la aplicación cumple el contrato [4] esperado por un servicio consumidor externo. Por último, las pruebas de extremo a extremo se encargan de comprobar si el sistema en su conjunto alcanza los objetivos previstos. En general, las pruebas de extremo a extremo se centran en comprobar el paso de mensajes entre los servicios, pero también podrían incluir comprobaciones que validen la correcta configuración de infraestructuras de red adicionales (por ejemplo, cortafuegos, *proxies*, equilibradores de carga) (De Angelis et al., 2021, p. 1).

Las pruebas unitarias, las pruebas de integración, las pruebas de microservicio único y las pruebas de extremo a extremo son suficientes para garantizar una cobertura de pruebas adecuada para las aplicaciones desplegadas y ejecutadas localmente, pero cada vez son más las aplicaciones que comienzan a desplegarse en la nube (Li et al., 2020, p. 3). Similarmente, Wu et al. (2022) comentan que, para probar microservicios, las técnicas de prueba de software predominantes incluyen pruebas unitarias, pruebas de integración, pruebas de rendimiento y pruebas de punto-a-punto. Lo cual sigue el modelo piramidal de prueba propuesto por Cohn (Wu et al., 2022, pp. 1-2).

Finalmente, los métodos de prueba entre pruebas de integración y pruebas unitarias se han convertido en una parte esencial. Limón [10] mencionó que las pruebas de integración y las pruebas de contrato son partes críticas para las pruebas de microservicios (Wu et al., 2022, p. 2).

#### 5.1.3.1 Pruebas unitarias

Las pruebas de microservicios comprenden a las pruebas unitarias, pruebas de servicios y pruebas de extremo a extremo [4]. Sin embargo, las pruebas unitarias se utilizan para identificar fallos en funciones o clases (Chen et al., 2023, p. 2).

En el caso de las aplicaciones de microservicios, las pruebas unitarias no difieren mucho de las de las aplicaciones monolíticas, en las que el objetivo de la prueba es una función, una clase, un método de una clase o una página web del *front-end* (Gong y Cai, 2023, p. 3).

La prueba unitaria consiste en escribir un pequeño fragmento de código para comprobar si se puede ejecutar una pequeña función del programa de destino [5]. Esta función puede ser una clase de Java o una función del lenguaje C. Es necesario definir el límite de la unidad que se va a probar antes de realizar la prueba, la unidad se denomina programa "caja negra". En el servicio tradicional, para verificar si el comportamiento de la "caja negra" en el límite cumple las expectativas, los probadores introducen datos en la caja negra para verificar la corrección de la salida. Sin embargo, en el microservicio, el funcionamiento del programa de la "caja negra" a probar puede depender del soporte de otros microservicios. Por lo tanto, en el proceso de prueba unitaria, no sólo es necesario simplemente introducir varios parámetros en el programa de "caja negra". También simula funciones externas / microservicios para generar errores especificados, y luego verifica si la lógica de manejo de errores de la función cumple con las expectativas [6] (Li et al., 2020, p. 2).

#### 5.1.3.2 Pruebas de integración

La prueba de integración es una extensión lógica de la prueba unitaria, es decir, la combinación de dos o más unidades probadas en un submódulo y, a continuación, la combinación de estos submódulos para formar un "subsistema"[7]. El objetivo de la prueba es comprobar si estos submódulos pueden cooperar entre sí de la manera esperada, comprobar la comunicación e interacción entre ellos, verificar si la interfaz funciona correctamente y, a continuación, garantizar el funcionamiento estable de todo el subsistema (Li et al., 2020, p. 2).

La complejidad de las aplicaciones de microservicios requiere pruebas automatizadas centradas en las interacciones entre los servicios. Aunque esto es posible con las pruebas de extremo a extremo, son propensas a errores, frágiles, caras de escribir, lentas de ejecutar y requieren el despliegue de toda la aplicación. Las pruebas de integración son una alternativa a las de extremo a extremo, ya que su alcance es menor y requieren el despliegue de un número significativamente menor de servicios (Reile et al., 2022, p. 1).

Las pruebas de integración de microservicios constan de dos partes. La primera es verificar que el servicio bajo prueba pueda interactuar adecuadamente con otros servicios. El segundo es probar la interacción entre el servicio y la base de datos. El objetivo principal de las pruebas de integración para microservicios es confirmar si la comunicación entre servicios es normal, sin importar si los servicios externos son correctos. Para la prueba de integración de la interacción de la base de datos, el objetivo principal es garantizar que el microservicio pueda acceder a la base de datos correctamente. Además, los errores de red también deben considerarse en las pruebas de integración de microservicios, ya que el acceso a los servicios también puede verse afectado por errores de red (Gong y Cai, 2023, p. 4).

Fowler [1] afirma que al realizar pruebas de integración de un sistema que contiene microservicios, cada servicio puede considerarse como un componente y los servicios pueden agruparse utilizando el enfoque tradicional de pruebas de

integración [8]. Más específicamente, se utiliza un entorno de prueba o ensayo para integrar los microservicios probados individualmente [25] (Sotomayor et al., 2019, p. 3).

#### 5.1.3.3 Pruebas de componente

En las aplicaciones de arquitectura de microservicios, las pruebas de componentes se refieren a las pruebas de aceptación de servicios individuales para verificar la funcionalidad y otras características de calidad. Aunque las pruebas de componentes se realizan solo en servicios individuales, también es necesario considerar las dependencias con otros servicios. Por tanto, el diseño de las pruebas de componentes debe tener en cuenta estas dependencias. Idealmente, se han implementado los servicios externos de los que depende el servicio bajo prueba. En este punto, los servicios externos se pueden utilizar directamente para completar las pruebas del componente. Sin embargo, en la mayoría de los casos, los otros servicios de los que depende el servicio bajo prueba no están listos, por lo que es necesario considerar el uso de un servicio simulado para reemplazar estas dependencias (Gong y Cai, 2023, p. 4).

#### 5.1.3.4 Pruebas punto-a-punto

La comprobación de las interacciones de una aplicación de microservicio con los servicios backend sólo puede llevarse a cabo ejecutando pruebas de extremo a extremo o amplias pruebas de integración (Reile et al., 2022, p. 1). Las pruebas punto-a-punto (e2e) son la mejor manera de comprobar si todo el sistema puede funcionar bien una vez desplegado [11]. Mientras que las pruebas unitarias y las pruebas con datos simulados garantizan que partes del producto funcionen de forma aislada, la prueba e2e es la forma de comprobar la funcionalidad desde la ingesta de datos hasta la visualización (Lei et al., 2019, p. 4).

Sin embargo, las pruebas de extremo a extremo son frágiles, caras de escribir, lentas de ejecutar y requieren el despliegue de toda la aplicación [9] (Reile et al., 2022, p. 1).

#### 5.1.3.5 Pruebas de contrato

Para mejorar la eficacia de las pruebas colaborativas entre microservicios, se propone un método de prueba por contrato. Contrato se refiere al acuerdo entre el Consumidor del servicio y el Proveedor. Contrato Prueba basada en el contrato para verificar la cooperación entre consumidores y proveedores [8]. Mediante la prueba del contrato, el contrato se puede utilizar como estándar intermedio para verificar si el contenido proporcionado por el proveedor cumple con las expectativas de los consumidores. La prueba de contrato puede ayudar a los evaluadores a verificar la coordinación e interacción entre microservicios. Al concentrarse en verificar el contrato entre consumidores y usuarios, se pueden reducir significativamente los costos de las pruebas y mejorar la eficiencia de las pruebas (Li et al., 2020, p. 2).

Sundar [25] afirma que las pruebas de contrato son parte integral de las pruebas de microservicios e identifica dos tipos de pruebas de contrato: modelo de servicio

productor-consumidor, las pruebas de integración verifican el contrato de la interfaz del productor mediante la creación de simulacros o trozos del productor, replicando así el servicio a ser consumido; y las pruebas de contratos impulsadas por el consumidor se realizan por parte del productor verificando los diferentes contratos de consumo que deben ser atendidos (Sotomayor et al., 2019, p. 3).

#### 5.1.3.6 Pruebas de microservicio único

La prueba de microservicio único se refiere al aislamiento de todos los demás servicios y recursos de los que depende un servicio y a la perspectiva del "usuario" para ver si el servicio puede proporcionar el resultado esperado. En comparación con la prueba de integración, la prueba de microservicio único se centra en la función de todo el servicio, no solo en la comunicación y el almacenamiento externos (Li et al., 2020, p. 2).

#### 5.1.3.7 Pruebas A/B

Las pruebas A/B son un enfoque sistemático para comparar el uso de dos versiones de un sistema y determinar cuál de las dos variantes es la preferida según algún criterio. Más concretamente, una prueba A/B consiste en un experimento controlado aleatorizado en el que se asignan unidades experimentales (por ejemplo, usuarios) a una de dos variantes, denominadas A y B, que se espera influyan en alguna métrica de interés. Esta métrica, el criterio de evaluación global, proporciona una medida cuantitativa del objetivo del experimento (en forma de hipótesis). Para comparar las dos variantes, se suelen utilizar pruebas estadísticas de hipótesis. La elección de las pruebas (y su potencia) depende de la distribución (supuesta) de los datos (Quin and Weyns, 2022, p. 2).

Las pruebas A/B se han utilizado ampliamente, desde pruebas de elementos entre usuarios, pruebas de precios de productos, evaluación de recomendaciones personalizadas, pruebas de características de productos y, de forma más general, evaluación del impacto de los cambios realizados en productos y servicios de software. Los experimentos controlados en línea son hoy en día una práctica habitual y muy utilizada en la práctica [15] (Quin and Weyns, 2022, p. 2).

#### 5.1.3.8 Pruebas en la nube

La prueba en la nube puede ahorrar tiempo en la configuración del entorno y proporciona un conjunto completo de entornos de prueba, donde los evaluadores solo necesitan establecer algunos parámetros o proporcionar scripts de prueba simples para realizar pruebas en la nube de inmediato. Después de que el evaluador especifica la configuración del hardware, la pila de software y la topología de la red, solo le lleva unas pocas horas crear un nuevo entorno de prueba. Debido a que es una aplicación basada en red, cuando aparecen problemas durante la prueba, también puede obtener soporte remoto y rápido de los proveedores de servicios de prueba en la nube, y rara vez se estanca o incluso detiene la prueba (Li et al., 2020, p. 2020, p. 5).

#### 5.1.3.9 Pruebas de regresión

Las pruebas de regresión funcional verifican el sistema después de la modificación, asegurando que todas las características, flujos y funcionalidades del sistema estén funcionando como en la versión anterior, o según una especificación o expectativa de funcionalidad correcta. Ayuda con el aseguramiento de la calidad del sistema y ayuda a evitar cambios no deseados en el sistema causados por la prisa por realizar cambios (Smith et al., 2023, p. 1).

Para estas pruebas, el sistema podría compararse con la versión anterior, basándose en métricas mensurables como la tasa de uso de recursos, la tasa de fallas del sistema, el rendimiento del sistema, etc., y evitar el lanzamiento de la última versión, si es necesario [5] (Kargar y Hanifizade, 2018, p. 1).

Siempre que ocurre una modificación del servicio, se requieren pruebas de regresión para detectar posibles fallas introducidas en las modificaciones [5]. La estrategia común empleada en las pruebas de regresión es volver a ejecutar casos de prueba utilizados anteriormente (denominado conjunto de pruebas originales), es decir, la estrategia de volver a probar todo (Chen et al., 2023, p. 1).

Una variante efectiva de las pruebas de regresión son las pruebas ex-vivo, estas utilizan información extraída del campo para aumentar los casos de prueba existentes o producir nuevos casos de prueba que pueden usarse para validar actualizaciones de software de manera consistente con los usos de campo [21, 32] (Gazzola et al., 2020, p. 1).

## 5.2 Contexto de aplicación de pruebas

## 5.3 Propuestas de la literatura

### 5.3.1 Benchmarking

Detti et al. (2023) presentan µBench, una fábrica de software de código abierto que crea y ejecuta aplicaciones de microservicios de referencia [27]. Los investigadores que utilizan µBench pueden controlar propiedades clave de la aplicación generada, como la cantidad de microservicios involucrados; la cantidad y tipo de recursos (CPU, disco, red) que demandan los microservicios; la malla de servicios de la aplicación, es decir, el gráfico de dependencia entre microservicios; HTTP o gRPC como protocolos de comunicación entre servicios. Después de crear la aplicación de microservicio, µBench la ejecuta en un clúster de Kubernetes y exporta métricas de Prometheus por microservicio, como la latencia y el rendimiento. Se pueden realizar evaluaciones comparativas que incluyan un conjunto de microservicios por solicitud que sean aleatorios o basados en seguimiento.
µBench facilita la comprensión de las aplicaciones de microservicios, mejora la evaluación de las soluciones en la nube frente a las diferentes características que pueden tener estas aplicaciones y, por último, pero no menos importante, nos ha resultado útil también con fines educativos para mostrar las ventajas, problemas y desafíos de los microservicios. aplicaciones a los estudiantes (p. 2).

En paralelo, Sedghpour et al. (2023) proponen HydraGen, un generador de referencia de microservicios de código abierto que puede emular aplicaciones personalizables basadas en microservicios para casos de uso de servicios web. HydraGen permite una evaluación experimental sistemática y exhaustiva de las implicaciones de diversas topologías de aplicaciones y complejidades computacionales y entre servicios para el rendimiento y la escalabilidad de los mecanismos de gestión de recursos nativos de la nube [21]. Este generador de referencia incluye las perillas de sintonización necesarias para configurar sistemáticamente 1) la arquitectura topológica de la aplicación, 2) la complejidad computacional de cada microservicio, 3) las características del tráfico entre microservicios emparejados, y 4) características de implementación relacionadas con la colocación y escalabilidad de microservicios (p. 2). Aunado a esto, proveen una comparación de su propuesta con µBench.

El trabajo de Smith et al. (2023) presenta un conjunto de prueba que puede utilizarse eficazmente como punto de referencia para la investigación sobre pruebas de software en sistemas basados en microservicios. Su enfoque consideró tanto las pruebas de regresión funcional como las pruebas de carga. Seleccionando dos sistemas de microservicios bien establecidos para crear un punto de referencia de prueba para brindar a la comunidad. Este doble enfoque para evaluar el rendimiento de los sistemas de microservicios ayuda a resaltar áreas clave de falla dentro de estos sistemas. El aspecto de prueba de carga de su estudio de caso destaca áreas dentro de los puntos finales que podrían causar un cuello de botella o falla dadas diferentes cantidades de carga. Comprender cómo responde el sistema a diversas cantidades de carga puede ayudar a los diseñadores a comprender áreas para ayudar a mejorar la experiencia del usuario. Su segundo aspecto de prueba implica analizar el sistema mediante pruebas de regresión funcional. Específicamente dentro de este enfoque, utilizaron pruebas automatizadas del sistema web para garantizar que el sistema se muestre de manera correcta y consistente en varios entornos de navegador (p. 7).

En este contexto, van Hoorn et al. (2018) describen la visión y el estado actual de su enfoque ORCAS para la evaluación comparativa de resiliencia eficiente de arquitecturas de microservicios. El enfoque explota el conocimiento sobre la relación de los patrones de resiliencia, los antipatrones y las inyecciones de fallas adecuadas, así como la arquitectura del sistema para generar experimentos de resiliencia, combinando simulaciones y puntos de referencia a nivel de prueba/producción. ORCAS se puede utilizar para detectar antipatrones de resiliencia y evaluar la efectividad de los patrones de resiliencia actuales. El beneficio esperado es que el número de pruebas comparativas que se ejecutarán con respecto al sistema real se reduzca considerablemente y al mismo tiempo se logren resultados comparables o mejores (p. 1).

### 5.3.2 Detección de anomalías y análisis de la causa raíz

Amaral et al. (2022) proponen un marco de prueba de rendimiento que aprovecha las métricas ocultas del kernel de BPF para localizar las causas fundamentales de la regresión del rendimiento. Su herramienta, MicroLens, es independiente de las aplicaciones y no requiere instrumentación de código intrusivo. El marco aplica un análisis multinivel, (i) construye un gráfico de atributos con las solicitudes de microservicios, (ii) califica los servicios para identificar las rutas críticas y (iii) clasifica las métricas de bajo nivel para resaltar la causa raíz de la regresión de desempeño. MicroLens correlaciona los síntomas de servicios problemáticos más probables con la utilización de recursos relevantes para identificar cuellos de botella que tienen síntomas de anomalías de servicio no obvios.

Su evaluación experimental con microbenchmarks muestra que MicroLens tiene una sobrecarga baja al recopilar métricas de BPF, con un aumento de latencia de menos del 18 % cuando se utiliza nuestra herramienta de recopilación de métricas de BPF con cliente y servidor en el mismo host y un 9 % entre hosts para Memcached y Memslap. Esta sobrecarga es muy baja en comparación con el estado-del-arte. Por ejemplo, Istio tenía un 120 % más de latencia en el mismo host y un 54 % más de latencia entre hosts. (pp. 2-9).

Camilli et al. (2022) diseñaron un enfoque de caja negra que no requiere disponibilidad de código y es capaz de detectar automáticamente problemas de rendimiento y confiabilidad a nivel de tipos de solicitudes de microservicios individuales. Explota las especificaciones API para determinar entradas de solicitudes válidas e inválidas, construye un modelo de Markov, toma muestras aleatorias del espacio de entrada particionado y, finalmente, calcula y visualiza el rendimiento y la confiabilidad en combinación. El enfoque genera y ejecuta pruebas ex-vivo de una manera no intrusiva, sin instrumentar un marco específico para aislar y probar microservicios individuales, como se propone en [27]. El enfoque se puede integrar en el entorno de prueba de una canalización de DevOps, lo que permite obtener comentarios rápidos sobre los problemas descubiertos (p. 2).

Chen et al. (2020) proponen Virtual War Room (VWR), un marco con simulación de arquitectura de microservicios, inyección de fallas y visualización, cuyo objetivo es ayudar a los investigadores a probar y comparar sus algoritmos de operación inteligente en múltiples tipos de sistemas de microservicios a gran escala. Con VWR, es conveniente validar sus algoritmos para la detección de anomalías y el análisis de la causa raíz. Simulamos tres arquitecturas clásicas y utilizamos los datos de rastreo producidos para realizar diferentes experimentos con el fin de validar la efectividad de los datos de rastreo producidos. Introducimos un algoritmo de detección de anomalías basado en Matrix Sketch, que es un algoritmo no supervisado y funciona bien en datos de transmisión de alta dimensión. Comparamos este algoritmo con dos algoritmos populares de detección de anomalías, a saber, IForest y 1SVM-RBF. Descubrimos que el algoritmo de detección de anomalías basado en Matrix Sketch funciona bien cuando la tasa de anomalías es aproximada a la tasa de anomalías real en sistemas del mundo real.

Además, encontramos que este algoritmo es sensible a la arquitectura y podemos mejorar su rendimiento utilizando menos funciones (p. 13).

Gazzola et al. (2021), por su parte, desarrollaron la herramienta ExVivoMicroTest aplicable a la arquitectura de microservicios. ExVivoMicroTest logró reproducir correctamente las interacciones que se observaron mientras se ejecutaba PiggyMetrics, y creó instancias y utilizó adecuadamente los simulacros que respaldan el aislamiento del servicio bajo prueba durante el procedimiento de de las pruebas de regresión. Sus resultados confirmaron que ExVivoMicroTest puede revelar fallas de regresión. En nuestros experimentos, reveló fallas tanto reales como inyectadas manualmente (p. 8).

Jin et al. (2020) proponen un método de localización de la causa raíz basado en el seguimiento de ejecución para MSA. En primer lugar, describen el seguimiento de ejecución del procesamiento de solicitudes mediante árboles de invocación y construimos una biblioteca de árboles estándar utilizando el algoritmo de coincidencia de árboles estándar. Luego, extraen las cadenas de invocación en el periodo de falla de seguimiento de ejecución parcial para que coincidan con la biblioteca de árbol estándar, y vectorizan la matriz de los nodos del árbol estándar que consume mucho tiempo para construir una matriz que consume mucho tiempo. Utilizan el algoritmo RPCA para analizar la matriz anómala que requiere mucho tiempo y el algoritmo de detección de anomalías de indicador único para localizar las causas fundamentales. Finalmente, generan nodos con puntuaciones anómalas altas. Los resultados experimentales muestran que el algoritmo propuesto clasifica las causas raíz entre las dos primeras con más del 90% de precisión para datos de muestra y datos de prueba dados, lo que verifica su efectividad (p. 10).

Karn et al. (2022) presentan una metodología automatizada de pruebas y resiliencia para la arquitectura distribuida de la aplicación. Utilizando una malla de servicios con la herramienta Istio, esto para monitorear el seguimiento entre microservicios para desarrollar pruebas automatizadas y de resiliencia. Se utiliza una herramienta de prueba de carga Locust para crear usuarios virtuales para pruebas de carga. El enlace de microservicio defectuoso se localiza a través de varios paneles de Istio. Se utilizan tres tipos de mecanismos para la resiliencia o corrección de la falla, incluido el escalado, la conmutación por error y el disyuntor. La configuración de prueba y resiliencia se puede utilizar de manera efectiva para la resolución de problemas de la red y la medición del rendimiento de cualquier aplicación en la nube durante fallas y congestiones de tráfico (p. 8).

Kohyarnejadfard et al. (2022) postulan un marco de trabajo para detectar anomalías y regresiones en ambientes de microservicios tomando ventaja de NLP y herramientas de código abierto como LLTng y Trace Compass. La estrategia general basada en NPL ayuda a distinguir patrones normales y anormales en una secuencia de eventos. Además de localizar anomalías, su marco propuesto también permite a los analistas ampliar las partes anómalas detectadas del rastro para descubrir la causa raíz de los problemas (p. 2).

Li et al. (2021) propone TraceRCA, un enfoque práctico de localización de microservicios de causa raíz mediante análisis de seguimiento, que se compone de detección de anomalías de seguimiento, extracción de conjuntos de microservicios sospechosos y clasificación de microservicios. La idea clave de TraceRCA es que un microservicio con más rastros anormales y menos rastros normales que pasan es más probable que sea la causa raíz.

IntelliFT es la propuesta de Long et al. (2020) para prueba de fallas automatizada para aplicaciones basadas en microservicios, que puede explorar el espacio de fallas de AUT de manera efectiva para exponer errores graves en el manejo de fallas (p. 8).

Pei et al. (2023) proponen PerfMS, una técnica de creación de perfiles basada en algoritmos genéticos para exponer las anomalías de rendimiento y detecta el posible cuello de botella en el rendimiento agregando y comparando rastros normales/anormales en microservicios (p. 6).

Pietrantuono et al. (2018) presentan un método de prueba en tiempo de ejecución para la evaluación bajo demanda de la confiabilidad en la arquitectura de microservicios. Los resultados sugieren que tanto la adaptabilidad en tiempo de ejecución al perfil real observado y al comportamiento de falla como la adaptabilidad en tiempo de prueba implementada por el nuevo algoritmo MART (que permite detectar fallas con pocas pruebas y al mismo tiempo preservar la imparcialidad de la estimación) son buenos puntos de partida para elaborar más en el futuro (p. 9).

Pina et al. (2018) muestran un enfoque para el monitoreo de microservicios, sin instrumentación ni intrusión en el sistema bajo prueba. Su objetivo era analizar los límites de un enfoque de caja negra, utilizando solo algunos de los módulos de infraestructura ya implementados en una arquitectura de microservicio. Recurrieron a la infraestructura de Netflix, para recopilar datos sin procesar de las invocaciones de microservicios. Los resultados muestran que su solución implica esfuerzos mínimos de configuración para integrarse en el sistema y producir información relevante para los administradores de un sistema o aplicación (p. 8).

Wang et al. (2020) propusieron un enfoque y sólido de localización de métricas de causa raíz mediante la incorporación de detección de anomalías de registros. Su enfoque consta de dos partes: recopilar puntuaciones de anomalías mediante el algoritmo de detección de anomalías de registro de última generación e identificar la métrica de la causa raíz mediante un análisis de correlación sólido con aumento de datos. Amplios experimentos en un sistema de microservicio de referencia demuestran que su enfoque puede localizar la métrica de la causa raíz con precisión en comparación con los métodos de referencia existentes y lograr un tiempo de respuesta corto. Por lo tanto, el enfoque puede ayudar a los ingenieros a ahorrar mucho tiempo y esfuerzo para diagnosticar y mitigar fallas lo antes posible (p. 8).

Wu et al. (2021) presentan un sistema automatizado de diagnóstico de rendimiento denominado MicroDiag para sistemas de microservicios. MicroDiag infiere causas

fundamentales detalladas a partir de métricas, lo que indica no solo el componente defectuoso sino también las pistas de la anomalía del componente defectuoso, al modelar la propagación de anomalías entre métricas con un gráfico de causalidad. Para lograr el gráfico de causalidad de las métricas, MicroDiag identifica primero la propagación potencial entre componentes y luego infiere las relaciones causales entre las métricas de componentes interdependientes con técnicas de inferencia causal. Como las propiedades de propagación de anomalías entre métricas son diferentes, se utilizan dos tipos de métodos de inferencia causal. Evaluaron su sistema en un banco de pruebas que se ejecuta en el motor de la nube de Google donde se implementa un punto de referencia de microservicio en un clúster de Kubemetes y se inyectan dos tipos de problemas de rendimiento. Los resultados experimentales muestran que MicroDiag puede clasificar el 97 % de las métricas culpables en una de las 3 causas más probables y supera al menos el 31,1 % de métodos del estado del arte (p. 6).

Xie et al. (2023) proponen ImpactTracer para resolver el problema de resolución de problemas en microservicios desde la perspectiva del modelado de propagación de fallas. ImpactTracer aplica Isolation Forest en indicadores de oro comerciales para detectar anomalías, lo cual es más preciso y adaptable que los métodos tradicionales. Para identificar el microservicio de causa raíz, ImpactTracer construye un gráfico de impacto para modelar completamente las rutas y probabilidades de propagación de fallas, sentando las bases para el algoritmo de seguimiento hacia atrás que atraviesa exhaustivamente el gráfico para rastrear el nodo de causa raíz. Amplios experimentos y resultados con datos del mundo real demuestran la eficacia y superioridad de ImpactTracer en la localización de culpables en microservicios (p. 6).

Yang et al. (2022) proponen WinG, una herramienta para manejar la heterogeneidad de la arquitectura del sistema y las invocaciones variables entre servicios, propusimos un sistema. Con el objetivo de localizar las causas fundamentales en los microservicios basados en contenedores. En primer lugar, el sistema evalúa el grado de anomalía de los elementos basándose en los vectores de características deformadas de las invocaciones, y luego localiza el grupo anómalo y señala al culpable con un módulo de filtrado de falsas alarmas. Los resultados experimentales en un conjunto de datos AIOps realista mostraron que WinG logra una precisión del 87%. Además, el módulo de filtrado de falsas alarmas de nuestro algoritmo se puede implementar con otros algoritmos para mejorar la precisión (p. 6).

### 5.3.3 Inyección o simulación de defectos

Bi et al. (2022) presentan VECROsim1, un sistema versátil de simulación de fallas de microservicio orientado a métricas y lanzar el conjunto de datos de referencia VECROsim. VECROsim funciona con delicadeza como un práctico conjunto de herramientas para generar conjuntos de datos de métricas de rendimiento anormales de sistemas de microservicios bajo demanda. Puede implementar automáticamente un sistema de microservicio personalizable con topología con un inyector de fallas rico en escenarios y un generador de carga del lado del usuario.

VECROsim también permite la recopilación integral de métricas de rendimiento y la generación automatizada de conjuntos de datos, registrando el ciclo de vida del sistema simulado (p. 1).

### 5.3.4 Estrategias de prueba

De Angelis et al. (2021) describen la propuesta de su metodología para realizar pruebas de regresión más efectivas, buscando evitar estrategias de pruebas que involucren probar todo el sistema (p. 10).

Por su parte, Ding et al. (2020) presentan un método de generación automática de datos de prueba para una solución de prueba de aplicaciones de microservicio. Este método amplía el archivo WADL de la interfaz de microservicio y genera automáticamente datos de prueba con diferentes parámetros de complejidad. En términos de parámetros combinados, se presenta AETG_I, el algoritmo por pares mejorado. Según los resultados experimentales, este método puede cubrir más del 95% de los errores incrustados previamente y reducir el recuento de datos de prueba del algoritmo AETG en al menos un 22%. Esto demuestra que el método puede generar menos datos de prueba bajo la condición de garantizar el efecto de la prueba, lo que contribuirá a mejorar la eficiencia de las pruebas y mejorará la capacidad de prueba del software de la empresa china SGCC (p. 4).

En su trabajo, Duan et al. (2021) desarrollaron un marco básico para pruebas automatizadas de aplicaciones de microservicios. Sobre esta base, introducen el algoritmo del árbol de decisión para clasificar y seleccionar automáticamente diferentes tipos de casos de prueba, lo que puede ampliar el alcance de la clasificación de los casos de prueba y al mismo tiempo garantizar la precisión de la clasificación de los casos de uso. Finalmente, el experimento demostró el rendimiento del modelo (p. 4).

Elsner et al. (2022) proponen microRTS, una técnica RTS dinámica para sistemas basados en microservicios. Al combinar la infraestructura de seguimiento distribuida establecida con instrumentación de código, microRTS recopila seguimientos de ejecución por prueba a nivel de método o clase en todos los servicios y clientes, lo que permite la selección de pruebas para pruebas de extremo a extremo automatizadas o manuales. Además, presentan un estudio de caso sobre RTS para pruebas manuales de un extremo a otro en CWA, un sistema basado en microservicios del mundo real. Sus resultados iniciales muestran que, si las pruebas manuales se especifican de forma bastante gruesa, microRTS no puede proporcionar ningún beneficio sobre la repetición de pruebas todas. Sin embargo, cuando eliminaron los seguimientos de ejecución por prueba utilizando el conocimiento del dominio, se excluyen hasta el 50% de los casos de prueba (p. 4).

Asimismo, en el trabajo de Gartziandia (2021) se demuestra a través de su enfoque es posible reducir el tiempo y esfuerzo de implementación de la automatización de pruebas de regresión para detectar problemas de rendimiento en actualizaciones de software (p. 3).

Giamattei et al. (2022) publican su trabajo en torno a una estrategia de caja gris específica para microservicios, la cual adopta una técnica de generación de pruebas combinatorias, respaldada por una herramienta automatizada (MACROHIVE), implementada como una colección de microservicios y que no requiere ejecutar sesiones de prueba separadas para cada microservicio. A diferencia de las herramientas existentes, la estrategia propuesta permite calcular la cobertura de los microservicios internos y proporciona información sobre el comportamiento fallido (p. 2). Su estudio de caso muestra que el enfoque propuesto es muy eficaz para detectar diferentes tipos de fallas (fallas de borde, fallas internas, fallas propagadas y fallas enmascaradas), al explorar dependencias internas y proporcionar información útil sobre microservicios defectuosos (pp. 2-10).

Del mismo modo, Gong y Cai (2023) plantean modelo de calidad para aplicaciones de arquitectura de microservicios y analiza las similitudes y diferencias con el modelo de calidad de productos de software ISO/IEC/IEEE 29119 en los estándares nacionales. También presentan los métodos de prueba para aplicaciones de arquitectura de microservicios (p. 5).

En el trabajo de Kargar y Hanifizade (2018) se propone un método que automatiza las pruebas de regresión orientado a la entrega continua (CD) en aplicaciones basadas en microservicios (p. 2).

Lei et al. (2019) analizan cómo emplear las herramientas disponibles con Kubernetes para realizar pruebas de rendimiento. Las principales contribuciones de este artículo incluyen su reveladora discusión sobre los factores que podrían producir cuellos de botella en el desempeño desde la perspectiva de la programación de recursos, así como métricas para evaluar el desempeño de un extremo a otro (p. 6).

Continuando, Li et al. (2020) presentan un sistema de prueba que aplica su propuesta de extensión de la estrategia piramidal clásica de prueba. También aclara los requisitos de uso de las herramientas de prueba en cada aspecto y explica el método de implementación de la estrategia de automatización de pruebas. Proporciona garantía de prueba para el desarrollo de aplicaciones de microservicios de la empresa china The State Grid (p. 2).

Lin et al. (2020) discuten sobre el entones estado actual de las pruebas en microservicios, para presentar un conjunto completo de soluciones técnicas de plataforma de prueba simulada (*mock*) para respaldar la interfaz simulada de microservicios en el desarrollo y las pruebas. La combinación y operación simultánea de servicios reales y servicios simulados mejoran en gran medida la eficiencia, la flexibilidad y el rendimiento del desarrollo y prueba de aplicaciones de microservicios (p. 5).

Ma et al. (2023) disertan sobre dos enfoques para la simulación de Kafka. El primer método emplea una instancia de Kafka en memoria (usando EmbeddedKafka5),

mientras que el segundo método emplea contenedores de prueba 6 (basados en Docker). EmbeddedKafka se puede ejecutar directamente en un entorno de desarrollo típico, mientras que los contenedores de prueba deben ejecutarse en Docker. El inicio de EmbeddedKafka (que utiliza una instancia de Kafka en memoria) requiere de decenas a cientos de milisegundos, mientras que el inicio de contenedores de prueba puede variar de segundos a decenas de segundos (según las especificaciones del hardware). Ambos métodos automatizan la limpieza y restauración de los cambios de datos realizados durante las pruebas; sin embargo, el rendimiento de EmbeddedKafka es generalmente mejor. En conjunto, recomendamos utilizar EmbeddedKafka para realizar pruebas unitarias para microservicios controlados por eventos (EDD) (p. 2).

En el trabajo de Meng et al. (2021) se propone un método de verificación para servicios de aplicaciones industriales que se ejecutan en plataformas de Internet industrial, las cuales generalmente se construyen a partir de microservicios. AITM se utiliza para caracterizar las propiedades estáticas y dinámicas del sistema. STG se utiliza para construir el modelo estándar del sistema. Se utiliza un algoritmo de verificación automática para verificar si la función de AITM es consistente con el documento de diseño. Entonces se determina la credibilidad del sistema. El método adopta tecnología de verificación formal, reemplaza el gráfico dirigido con una estructura de árbol y reduce la complejidad computacional (p. 5).

Quenum y Aknine (2018) presentan un enfoque para las pruebas automatizadas de microservicios. Después de definir los contextos acotados según el enfoque DDD, identifican los servicios que proporcionarán la funcionalidad requerida. Para ceñirse a escenarios de prueba relevantes y limitar la ambigüedad en el sistema, recomiendan generar las pruebas a partir de especificaciones formales. Empleando agentes inteligentes como abstracciones para los servicios. Los objetivos perseguidos por los agentes se definen formalmente siguiendo una lógica temporal. Luego derivaron las pruebas (unitarias y de aceptación) de esas especificaciones. Para ilustrar sus ideas, utilizaron dos marcos, uno para pruebas unitarias (RSpec) y otro para pruebas de aceptación (Cucumber). Además, analizaron su método para obtener las pruebas a partir de las especificaciones formales (p. 7).

Quin y Weyns proponen SEAByTE, un artefacto novedoso que aplica la autoadaptación [29] para mejorar la automatización de las pruebas A/B para respaldar la evolución de los sistemas de microservicios. En contraste con los artefactos existentes que apuntan a enfoques novedosos para diseñar sistemas autoadaptativos, SEAByTE explota la autoadaptación como un medio para resolver una tarea clave de los ingenieros de software de sistemas basados en servicios: automatizar la experimentación continua (p. 6).

Además, Rattanukul et al. (2023) exponen la función de Microusity, una herramienta que realiza pruebas API RESTful de microservicios BFF. La herramienta rastrea las solicitudes procesadas por BFF, crea un mapeo de solicitudes principales a solicitudes secundarias y proporciona informes de prueba, tanto en formato textual

como de visualización, para ayudar a los desarrolladores a rastrear y solucionar problemas. Este documento hace las siguientes contribuciones clave (p. 1):

- *Fuzzing* de API de BFF y seguimiento de solicitudes: un enfoque novedoso para asignar la solicitud que ingresa al BFF a las solicitudes enviadas a los microservicios de backend.
- Una visualización basada en gráficos para ayudar a los desarrolladores a comprender y depurar los errores causados por los microservicios backend.

Reile et al. (2022), implementaron y presentaron Bunk8s, una herramienta para pruebas de integración amplia de microservicios en Kubernetes. Bunk8s es independiente del marco de prueba utilizado para escribir o generar pruebas, independiente de la infraestructura o plataforma en la que se ejecuta Kubernetes y admite la publicación de resultados de pruebas. Bunk8s es ideal para desarrolladores que planean escribir un nuevo conjunto de pruebas, así como para desarrolladores que desean revisar y mejorar el conjunto de pruebas ya existente de una aplicación de microservicio (p. 5).

Schulz et al. (2019) describen dos algoritmos (adaptación basada en registros y basada en modelos) para generar modelos de carga de trabajo personalizados para microservicios para pruebas de carga. A diferencia de los enfoques existentes para la generación de modelos de carga de trabajo representativos, estos modelos de carga de trabajo personalizados permiten probar un determinado subconjunto de todos los microservicios de una aplicación de forma aislada. Por lo tanto, con respecto a los equipos de desarrollo de microservicios independientes, cada equipo de desarrollo puede probar su servicio sin implementar toda la aplicación. Los servicios que no se vean afectados por la prueba no se implementarán, lo que ahorrará los recursos necesarios para ejecutar la prueba. Además, nuestro enfoque está automatizado para simplificar la implementación en un proceso de entrega continua.

Su evaluación experimental con una aplicación de microservicio mostró que los algoritmos propuestos no solo permiten pruebas de carga eficientes en el uso de recursos, sino que también generan una carga de trabajo más representativa que un modelo de carga de trabajo simple basado en solicitudes. Además, podrían reducir el tiempo de ejecución de la prueba requerido en comparación con una prueba a nivel de sistema, especialmente para combinaciones de microservicios bajo prueba con una cantidad baja de puntos finales. El enfoque basado en modelos pareció superar al enfoque basado en registros en cuanto a representatividad y comprensibilidad de los modelos de carga de trabajo generados, aunque el enfoque basado en registros mejoró para un mayor número de puntos finales probados (p. 12).

Ünver y Britto (2023), propusieron Pomegranate, una herramienta que permite agilizar el escaneo de vulnerabilidades de seguridad en microservicios y recomienda refactorizaciones para abordar las vulnerabilidades identificadas (p. 10). Todo a través de la agrupación de herramientas populares para escanear vulnerabilidades.

Vassiliou-Gioles (2020) define un marco TTCN-3 definiendo un mapeo entre operaciones de API RESTful basadas en HTTP/JSON. Proporcionamos una implementación de referencia para TTworkbench de Spirent y la aplicamos a varias aplicaciones en varios dominios. Uno de los hallazgos clave fue que, al utilizar un enfoque de prueba estructurado, pudimos identificar especificaciones insuficientes, como códigos de estado no documentados para las respuestas. Si bien todas las API utilizaron el término API REST, según nuestra evaluación, ninguna de las API probadas alcanzó la madurez de nivel 3 de RMM. Ya que sólo una de ellas hizo algún uso de conceptos hipermedia.

Wang et al. (2019) proponen una solución de prueba de automatización de aplicaciones de microservicios y se diseña un marco de pruebas basado en la modularización y la tecnología basada en datos. El marco completa las pruebas automatizadas de la interfaz de microservicio mediante la coordinación entre la capa de adaptación, la capa de datos, la capa de casos de prueba, la capa de ejecución, la capa de análisis y la capa de gestión. La estructura de archivos WADL extendida y la estructura de datos de parámetros están diseñadas para la generación de datos de casos de prueba, que es la parte clave de la prueba de automatización, y también se optimiza el problema de cobertura de combinación de parámetros (p. 4).

Finalmente, Wu et al. (2022) proponen un método de verificación para microservicios controlados por eventos, llamado CCTS (Composite Contract Testing Service), para que los evaluadores realicen pruebas básicas en microservicios controlados por eventos y localicen errores potenciales. CCTS se basa en el modelo de estado especificado para describir las interacciones de transacciones de larga duración entre servicios y la prueba de contrato impulsada por el consumidor (CDCT) para verificar la exactitud de los mensajes. En CCTS, propusieron un método para detectar la posible ruta de ejecución para ayudar al evaluador a comprender las transiciones de eventos entre servicios en el proceso. También presentaron un método para analizar registros de eventos para determinar la exactitud de la interacción en escenarios reales y permitir que los evaluadores comprendan si la ruta de ejecución de mensajes reales correspondía a la secuencia correcta de registros de eventos, basándose en la detección de una ruta de ejecución potencial (p. 1).

## 5.4 Herramientas

### 5.4.1 Generales

Jaeger es una herramienta utilizada para mostrar los valores de métricas de rendimiento (Amaral et al., 2022, p. 1).

Memslap es una herramienta de evaluación comparativa y generación de carga de subprocesos múltiples para el conocido sistema de almacenamiento en caché de objetos de memoria distribuida. Memslap gestiona las conexiones de red con libevent de forma asincrónica con I/O sin bloqueo. Las dos métricas principales definidas por este marco son las transacciones por segundo (TPS) (es decir,

operaciones/segundo) y la media geométrica de la latencia de las solicitudes (Amaral et al., 2022, p. 5).
Prometheus es es un sistema de monitoreo popular para puntos finales de datos distribuidos y una base de datos de series temporales capaz de almacenar métricas recopiladas (Bi et al., 2022, p. 5).

Stress-ng10 una herramienta para cargar y estresar un sistema por medio de fallas de CPU. NetEm8 es una herramienta para provocar fallos a nivel de red. Por otro lado, Chaos Mesh 7 es una herramienta utilizada para inyectar varias fallas externas en el sistema implementado. Es una plataforma de ingeniería del caos (experimentación en un sistema distribuido para generar confianza en la capacidad del sistema para soportar condiciones turbulentas e inesperadas en producción) para Kubernetes (Bi et al., 2022, p. 7).

Para detectar anomalías, inferir la causa raíz y restaurar el servicio fallido lo antes posible, los operadores confían en sistemas de rastreo como X-trace [4], Zipkin [5] para perfilar y monitorear la arquitectura de microservicio. Estos sistemas de seguimiento pueden registrar la ruta de ejecución de cada solicitud. Con esta información se puede gestionar la complejidad de un sistema de microservicio (Chen et al., 2020, p. 1).

Heorhiadi et al. propone Gremlin [36], un marco para pruebas de resiliencia para evaluar la capacidad de un sistema de microservicio para recuperarse de fallas. Jindal et al. introduzca Terminus [37] para estimar la capacidad de un microservicio, definido como el número máximo de solicitudes de usuario procesadas con éxito por segundo, en diferentes configuraciones de implementación a través de pruebas de carga, y ajustando un modelo de regresión a los datos de rendimiento adquiridos (Camilli et al., 2022, p. 3).

La evaluación de rendimiento suele basarse en una secuencia predefinida de llamadas HTTP que imitan el comportamiento del usuario y son generadas por herramientas externas, como Jmeter, para las que proporcionan los archivos de prueba de entrada (Detti et al., 2023, p. 3).

WADL (lenguaje de descripción de aplicaciones web) es un tipo de documento que se utiliza para describir la interfaz de microservicio REST, incluidos los recursos del servicio, las relaciones de los recursos y los estilos de operación. Los ingenieros de pruebas pueden diseñar y crear casos de prueba analizando el archivo WADL y obteniendo información relevante. Sin embargo, el archivo WADL sólo representa una pequeña parte de los parámetros de la interfaz, por lo que es necesario ampliarlo debido a la falta de una base de prueba (Ding et al., 2020, p. 1).

A medida que los microservicios acceden a microservicios dependientes mediante API REST, cualquier dependencia se puede burlar fácilmente utilizando marcos como Wiremock.4 En teoría, no se requieren pruebas de integración ya que todas las interacciones entre microservicios se pueden burlar en función de su documentación API (Eismann et al., 2020, p. 4).

Por ejemplo, ELK [2] y Prometheus [9] son dos sistemas de monitoreo populares que se pueden usar para probar aplicaciones en la nube, mientras que Monasca [4], CloudHealth [42] y Varys [43] proporcionan funciones de monitoreo más sofisticadas. incluida la capacidad de encender y apagar sondas dinámicamente (Gazzola et al., 2020, p. 4).

Burp Suite se emplea para recopilar entradas y salidas de pruebas de caja negra [47] (Giamattei et al., 2022, p. 5).

PACT [9] es uno de los marcos más populares para pruebas de contratos de microservicios. Utiliza pruebas de contratos impulsadas por el consumidor (Gong y Cai, 2023, p. 4).

Las pruebas de un extremo a otro de aplicaciones de arquitectura de microservicios son menos diferentes de las pruebas de sistemas de aplicaciones monolíticas. También se puede implementar utilizando algunas grabaciones y reproducciones mediante herramientas de prueba de automatización, como Selenium[10], Robot Framework[11] (Gong y Cai, 2023, p. 4).

Diffy10, una herramienta desarrollada por Twitter a finales de 2016, la cual permite utilizar el tráfico de entrada como entrada para probar el microservicio desarrollado, lo que podría resultar en una reducción considerable de los costos de las pruebas. Esta herramienta ejecuta una prueba de regresión comparando los resultados de respuesta de los microservicios de la versión en ejecución en el entorno de producción con la nueva versión desarrollada a la que se puede acceder en el entorno de desarrollo (Kargar y Hanifizade, 2018, p. 4).

La malla de servicios Istio [2] es una herramienta de código abierto para implementar una aplicación y recopilar señales de monitoreo para capturar rastros de comunicación entre microservicios. Schermann et al. [12] han descrito en su trabajo que Istio es una herramienta líder para construir una malla de microservicios para ejecutar una aplicación y cumple con requisitos operativos complejos, como pruebas y enrutamiento de tráfico basado en proxy (Karn et al., 2022, p. 1).

Varias herramientas para probar (o cargar), por ejemplo, JMeter, Tsung, Gatling, Grinder, etc., están disponibles para probar cada microservicio de aplicación (Karn et al., 2022, p. 2).

Locust genera una gran cantidad de usuarios virtuales para agrupar diferentes microservicios de aplicaciones. El comportamiento de cada usuario virtual se describe en un script de Python (comúnmente llamado locustfile.py) y el proceso de swarming se monitorea en tiempo real desde una interfaz web. También permite especificar el tiempo de espera mínimo y máximo por usuario simulado entre la ejecución de tareas. A diferencia de otras herramientas de prueba de carga, Locust no utiliza devoluciones de llamada para cada usuario virtual [32] y utiliza un proceso ligero independiente (Karn et al., 2022, p. 4).

Protractor es un marco para pruebas de un punto-a-punto (Li et al., 2020, p. 4).

La herramienta Teamcity coloca todo el proceso de prueba en un proceso de prueba de integración periódica automatizado, desde el código de detección, la compilación y construcción, la ejecución de la prueba, el registro de resultados, las estadísticas de la prueba, etc., todo se completa automáticamente sin intervención manual [11, 12] (Li et al., 2020, p. 5).

Visual Studio Tests es un marco de trabajo para pruebas (Li et al., 2020, p. 5).

La tecnología de simulación internacional es JMockit, que se puede aplicar a una variedad de entornos de prueba (Lin et al., 2020, p. 2).

Chaos Monkey [2] se utiliza para inyectar fallas aleatoriamente en el sistema de producción para probar la resiliencia (Long et al., 2020, p. 1).

Hystrix [3] es una biblioteca resistente que proporciona patrones comunes de manejo de fallas para microservicios, como tiempo de espera, reintento y interrupción de circuito (Long et al., 2020, p. 6).

Existen algunos frameworks que ayudan a implementar el rastreo, como ZipKin [6], Opentracing [7] o Dapper [8] (Pina et al., 2018, p. 1).

Grafana [5] es una plataforma abierta para análisis y monitoreo, altamente flexible y personalizable (Pina et al., 2018, p. 3).

Apache JMeter es una herramienta de simulación empleadas en pruebas de carga (Pina et al., 2018, p. 7)

Vector [14], es un framework que permite la creación de paneles con métricas, como CPU o utilización de red. El módulo fuerza la existencia de un agente, llamado Performance Co-Pilot (PCP), en cada host o aplicación a monitorear (Pina et al., 2018, p. 7).

Con el desarrollo impulsado por el comportamiento (BDD), los evaluadores pueden definir escenarios de aceptación en lenguajes menos técnicos (lenguajes específicos de dominio) y ejecutarlos con el código real. Sostenemos que, en lugar de definir las pruebas a partir de la comprensión de las especificaciones del software por parte de los desarrolladores, pueden generarse a partir de especificaciones más formales del sistema (Quenum y Aknin, 2018, p. 1).

RSpec7 y Cucumber8 son dos de esos marcos que se hicieron muy populares en varios conjuntos de herramientas de desarrollo de software (por ejemplo, Ruby on Rails (rails9), Node.js10). RSpec automatiza las pruebas unitarias, mientras que Cucumber se centra en las pruebas de aceptación (Quenum y Aknine, 2018, p. 3).

Gatling es una herramienta popular para la generación de prueba de carga (Rangaiyengar et al., 2023, p. 4).

Zeek (anteriormente conocido como Bro) es una herramienta de monitoreo de red especializada en el registro de eventos y un potente manejo de eventos. Es extensible con su propio lenguaje de programación [24]. Zeek se puede utilizar para realizar análisis del tráfico de la red y crear alertas para el usuario mediante scripts (Rattanukul et al., 2023, p. 2).

Para las pruebas de integración de microservicios desde el interior del clúster k8s, existen varias herramientas como Octopus [10], Istio [11] y Jenkins [12] (Reile et al., 2022, p. 1).

Las pruebas JUnit se utilizan para pruebas automatizadas de navegadores web (Smith et al., 2023, p. 3).

Utilizamos el marco TestNG con la capacidad de paralelizar las pruebas [12]. Además, TestNG permite al evaluador tener más control sobre las pruebas y puede especificar la cantidad de subprocesos en los que se ejecutarán las pruebas. Pasamos de usar JUnit como nuestro marco de prueba a usar TestNG. Después de cambiar de marco, paralelizamos nuestras pruebas para hacerlas más eficientes. Después de la paralelización de las pruebas de Selenium, ahora el tiempo necesario para ejecutarlas se redujo drásticamente (Smith et al., 2023, pp. 3-4).

Otra herramienta interesante es Hoverfly [5], brinda soporte para simular la inyección de fallas en las comunicaciones hacia/desde una API, permitiendo escribir pruebas automatizadas para simular la comunicación con otros microservicios (Sotomayor et al., 2019, p. 2).

Las pruebas unitarias de microservicios se realizan utilizando marcos xUnit como JUnit o NUnit (Sotomayor et al., 2019, p. 2).

Las herramientas para respaldar las pruebas de contratos incluyen Pact [18] y Spring Cloud Contract [21] (Sotomayor et al., 2019, p. 3).

Las herramientas utilizadas para las pruebas del sistema incluyen Appium [10], Docker Compose Rule [11], Gatling [12], Grem-lin [13] y Selenium IDE [19], entre otras (Sotomayor et al., 2019, p. 3).

OWASP ZAP1 es una herramienta de código abierto para detectar vulnerabilidades en aplicaciones basadas en web mediante la realización de análisis dinámico. Nmap network scanner2 y Kubeaudit3 son ejemplos de herramientas de auditoría de seguridad que se pueden aplicar a microservicios implementados en clústeres de Kubernetes (Ünver and Britto, 2023, p. 1).

En la categoría de contenedores, se elige The Docker Bench for Security porque aplica toda la lista de verificación CIS Docker Benchmark [15], que se utiliza mucho

en la industria como guía. Trivy es una herramienta poderosa con funciones ricas y una gran base de datos de vulnerabilidades conocidas. También preferimos integrar una herramienta de escaneo estático como Terrascan para analizar en profundidad el código fuente y la configuración del contenedor antes de ejecutar aplicaciones en contenedores en producción. En la categoría Kubernetes, seleccionamos kube-bench y kube-hunter porque son herramientas líderes de escaneo de Kubernetes en términos de cobertura de las debilidades de seguridad identificadas, reputación de la comunidad y facilidad de instalación. En la categoría de seguridad de red, kube-hunter y Terrascan también cubren los defectos de esta categoría. Además de ellos, incluimos OWASP ZAP y Nmap (Ünver and Britto, 2023, p. 1).

TTCN-3, Notación de control de pruebas y pruebas, es el lenguaje de implementación y especificación de pruebas definido por el Instituto Europeo de Normas de Telecomunicaciones (ETSI) para la definición precisa de procedimientos de prueba para pruebas de caja negra de sistemas distribuidos. Se adapta bien a los requisitos de prueba de las aplicaciones basadas en RESTful con posibilidades de describir de manera eficiente comportamientos de prueba distribuidos complejos a través de estímulos y respuestas paralelos. Uno de los beneficios de TTCN-3 es que permite la especificación de pruebas de forma independiente del lenguaje y la plataforma, siguiendo así el paradigma de microservicio para elegir la tecnología que mejor se adapta a la implementación de un determinado. Nuestro objetivo es considerar un sistema de prueba TTCN-3 como un microservicio más, con objetivos muy específicos (Vassiliou-Gioles, 2020, pp. 2-3).

Las herramientas de prueba de automatización de interfaces de aplicaciones de microservicios incluyen WebInject, WStest, SoapUI, TestMaker, QEngine, JMeter, etc. [3][12] (Wang et al., 2019, p. 3).

Node-exporter3 se utiliza para monitorear el sistema operativo de los nodos del servidor; Cadvisor4 para monitorear los recursos de los contenedores y la malla de servicios Istio5 para monitorear las interacciones de red entre microservicios, incluidos los tiempos de respuesta (Wu et al., 2021, p. 3).

Las herramientas relacionadas, como JUnit, JMeter y Selenium, han sido bien aceptadas y ampliamente utilizadas en la industria (Wu et al., 2022, p. 1).

Spring Cloud Contract1 y Pact2 representan la mayor parte de la participación de mercado en el área de prueba de contratos. Spring Cloud Contract es parte del ecosistema Spring y no admite pruebas basadas en eventos. Pact es un marco de prueba de contratos de código abierto y admite diferentes idiomas famosos (Wu et al., 2022, p. 2).

### 5.4.2 Benchmark

DeathStarBench (DSB) [31] es un aplicación de código abierto para microservicios ampliamente utilizada para pruebas. Define tres aplicaciones de microservicios en la nube: red social, servicio de medios y reserva de hotel. El banco de pruebas de

redes sociales es el ejemplo más complejo y combina el servidor web Nginx como punto de entrada para varios microservicios. Cada microservicio proporciona una funcionalidad específica para interactuar con el usuario mientras lee o crea una publicación. El benchmark se compone de una fase de inicialización para registrar usuarios en la red social. Por tanto, dependiendo de la acción del usuario, se activará un conjunto diferente de microservicios (Amaral et al., 2022, p. 5).

TrainTicket es un benchmark de código abierto cuya función es la reservación de boletos de tamaño mediano (41 microservicios) que admite el registro y seguimiento de aplicaciones y viene con 22 casos de fallas comprobables informados por ingenieros de software [34]. Cada caso de falla en TrainTicket se implementa modificando manualmente el código fuente de los servicios afectados (Bi et al., 2022, p. 11; Giamattei et al., 2022, p. 4).

m-Ticket es un sistema de tickets multifinales basado en SpringBlade que proporciona servicios de venta de entradas en diversos campos, como transporte, alojamiento, atracciones turísticas y películas, y apoya la gestión, seguimiento y localización de servicios (Chen et al., 2023, pp. 12-13).

z-Shop es un sistema de centro comercial orientado a dispositivos móviles que proporciona servicios de gestión integral para productos, tiendas, promoción de contenidos, pedidos, logística, entre otros (Chen et al., 2023, p. 13).

JOA es un sistema ofimático basado en Spring Cloud que proporciona visualización integral de información, circulación de documentos, aprobación de procesos, gestión de planes, gestión de personal de la organización, gestión de contratos, gestión de fondos y servicios de gestión de materiales para la organización con múltiples departamentos y niveles secretos (Chen et al., 2023, p. 13).

Teastore [23] es una aplicación de seis microservicios que implementa un sitio web de comercio electrónico que vende productos relacionados con el té (Detti et al., 2023, p. 2).

Hay otras aplicaciones de microservicio disponibles, como la aplicación de demostración de Google Online Boutique [29], Bookinfo [26], JPetStore [30], PetClinic [31]. Aunque muchos de ellas no están destinados a ser aplicaciones de evaluación comparativa de investigación, sino aplicaciones con fines de demostración (Detti et al., 2023, p. 2).

Las aplicaciones orientadas a microservicios, por ejemplo, las aplicaciones Sock Shop5 o Tea Store6, pueden considerarse como un punto de partida para crear dichas aplicaciones de referencia, junto con la plataforma CASPA [15], que puede proporcionar carga de trabajo y monitoreo (Eismann et al., 2020, p. 10).

Hipster Shop es una aplicación de comercio electrónico basada en web de 10 niveles donde los usuarios pueden buscar, agregar y comprar productos (Karn et al., 2022, p. 2).



## 5.5 Retos

Los microservicios han traído consigo nuevos desafíos. La complejidad de la intercomunicación que estaba oculta dentro de las aplicaciones monolíticas se ha trasladado a la capa de red con microservicios, aumentando la presión [4] e introduciendo problemas en cascada debido a dependencias complejas [5].
Como resultado, determinar la causa raíz de la regresión del rendimiento de los microservicios se ha vuelto un desafío. Los principales obstáculos son (1) las implicaciones topológicas en cascada para el rendimiento entre microservicios, que ocultan el origen del problema; (2) la falta de conocimiento sobre las fases de la aplicación puede conducir potencialmente a una detección de servicios críticos falsamente positiva; y (3) la utilización de recursos del servicio es un indicador imperfecto del rendimiento de la aplicación [5]–[7], lo que podría generar falsos positivos (Amaral et al., 2022, p. 1).

Desafortunadamente, la mayoría de los incidentes en los sistemas de nube comerciales no están disponibles públicamente. Como resultado, los investigadores han puesto un esfuerzo considerable en el desarrollo de varios sistemas de microservicios experimentales [13], [14] para generar métricas de anomalías simuladas. Sin embargo, estas herramientas no pueden refactorizar rápidamente su funcionalidad, escalar la arquitectura y personalizar las características de las fallas. La falta de conjuntos de datos sobre fallas de alta calidad ha disuadido a muchos investigadores de dedicar sus esfuerzos al problema (Bi et al., 2022, p. 1).

Si bien las pruebas de rendimiento [8, 9] y las pruebas de confiabilidad [10, 11] de sistemas de microservicios se han estudiado como problemas separados, existe una sorprendente falta de enfoques de prueba integrados capaces de evaluar los dos atributos de calidad y derivar relaciones mutuas (Camilli et al., 2022, p. 2).

La escalabilidad y el rendimiento se encuentran entre los atributos de calidad de los microservicios que plantean la mayoría de los desafíos de las pruebas [14]. La industria de TI considera las pruebas de rendimiento como el principal problema, lo que exige mayores esfuerzos de investigación [15, 16]. Generar pruebas apropiadas para revelar problemas relacionados con estas cualidades es un desafío, ya que el espacio de entrada suele ser grande y las condiciones operativas suelen ser inciertas [17, 18]. Por lo tanto, intentar todas las combinaciones posibles de valores de entrada en todas las condiciones posibles no es práctico y, en muchos casos, incluso inviable. Por ejemplo, sólo unos pocos valores de entrada pueden detectar problemas en el rendimiento [19], y encontrar esos valores es principalmente una actividad manual, intelectualmente intensiva y laboriosa [20] (Camilli et al., 2022, p. 2).

Si ocurre una falla en una arquitectura de microservicio, es más difícil para los operadores localizar las causas fundamentales debido a la creciente complejidad de las dependencias del servicio. Cualquier problema en un servicio, como una falla en el disco duro o una desconexión de la red, puede causar que el sistema de microservicio se averíe (Chen et al., 2020, p. 1).

Los investigadores necesitan implementar sus propios puntos de referencia de microservicios e inyectar algunas fallas en estos sistemas para detectar casos anormales. De lo contrario, necesitan algunos datos de desempeño existentes proporcionados por empresas comerciales o comunidades. Sin embargo, este enfoque plantea algunos desafíos para los investigadores (Chen et al., 2020, p. 2):

- Muy pocos puntos de referencia de microservicios. Siempre es necesario contar con puntos de referencia de microservicios para validar la eficiencia de los métodos de operación inteligentes. Sin embargo, existen muy pocos sistemas de evaluación comparativa de microservicios de código abierto, excepto algunas arquitecturas simples, como Sock-shop [12]. A los investigadores les lleva mucho tiempo e incluso es inviable diseñar e implementar sus propios sistemas de microservicios a escala.
- Un coste elevado en el rastreo distribuido. Una gran escala
- El sistema de microservicios siempre comprende numerosos servicios interconectados. Para operar y comprender su comportamiento, se ha utilizado como módulo predeterminado un seguimiento distribuido que registra las rutas de ejecución de cada solicitud. Sin embargo, el sistema de seguimiento ether requiere instrumentación de código en la aplicación y modificaciones del entorno de ejecución, lo que significa que los investigadores deben estar familiarizados con los códigos de referencia y de instrumento por sí mismos.
- Dificultad en la inyección de fallas. En realidad, los casos de
- Los fallos y anomalías son menos frecuentes que los casos normales. Para recopilar más casos anormales, siempre confiamos en la inyección de fallas para imitar fallas. Por lo tanto, se necesitan algunas herramientas de inyección de fallas. Existen algunos métodos y herramientas para que los investigadores lleven a cabo un ingeniero del caos como Netflix ChaosMonky [13], [14] e Istio [15]. Sin embargo, sigue siendo una tarea molesta en los sistemas de microservicios debido a su escala y complejidad.
- Un gran volumen de datos de seguimiento del caos. aunque muchos
- Los sistemas de microservicios existentes en el mundo real han recopilado un gran volumen de datos de rastreo, estos datos de rastreo son siempre un caos con muchos ruidos y errores, como datos faltantes, secuencias de eventos desordenadas y valores de medición incorrectos. Debido a este caos, no es trivial encontrar algoritmos eficientes para operar sistemas de microservicios. Además, la mayoría de los datos de rastreo recopilados no están etiquetados como "normal" o "anormal". Estos datos se recopilan en forma continua y deben procesarse a tiempo.

En las pruebas de regresión, la estrategia de volver a probar todo podría no ser aceptable en el caso de un sistema de microservicios con una gran cantidad de servicios implementados e iteraciones rápidas (Chen et al., 2023, p. 1).

Si un investigador quiere probar sus hallazgos para aplicaciones más grandes, por ejemplo, creadas por cientos de microservicios, o trabaja en una empresa que realmente tiene este tipo de aplicaciones a gran escala (ver Netflix, Google) o

simplemente no puede. En general, muchos investigadores en esta área podrían tener posibilidades limitadas de evaluación comparativa (Detti et al., 2023, p. 2).

En comparación con la arquitectura tradicional, la aplicación de microservicio se ejecuta en un sistema distribuido, lo que significa que cada servicio puede ejecutarse en diferentes entornos y tener dependencias complejas entre sí. Esto aumenta enormemente la dificultad de las pruebas, de modo que las pruebas manuales tradicionales no pueden cumplir completamente con los requisitos de los microservicios [2] (Ding et al., 2020, p. 1).

La investigación sobre las pruebas de aplicaciones de microservicios, incluidos los métodos de prueba [4], las herramientas de prueba, los estándares y mecanismos de evaluación, aún se encuentra en la etapa exploratoria. Además, las pruebas eficientes del sistema son cada vez más importantes [5]. En el proceso de prueba, diferentes casos de prueba de software contienen mucha información de atributos y cada caso de prueba es una prueba para diferentes situaciones. Para seleccionar casos de prueba que cumplan con el alcance de la prueba, es necesario examinar una gran cantidad de casos de prueba. Si se basa en la selección manual, es difícil garantizar que la cantidad y calidad de los casos de prueba seleccionados sean las mejores, y dicho proceso operativo requiere mucho tiempo de trabajo para completarse, por lo que las pruebas automatizadas surgen en el momento histórico [6] [7] (Duan et al., 2021, p. 1)

Las pruebas de rendimiento ya se consideran un desafío en los sistemas tradicionales [30]. Peor aún, los cambios arquitectónicos, tecnológicos y organizativos inducidos por los microservicios también tienen un impacto en las prácticas de ingeniería del rendimiento [21] (Eismann et al., 2020, p. 2).

El proceso de evolución de los microservicios requiere probar internamente cada nueva versión del servicio antes de que el servicio actualizado pueda implementarse en el campo. Sin embargo, las pruebas internas son inevitablemente parciales y a menudo no logran validar escenarios que representen cómo se utilizan realmente los servicios en el campo. De hecho, los casos de prueba (por ejemplo, casos de prueba de unidad, integración y sistema) son diseñados por desarrolladores sin conocer exactamente ni las características del entorno operativo ni el comportamiento de los usuarios (Gazzola et al., 2020, p. 1).

Existe una investigación limitada sobre modelos de calidad y métodos de prueba para arquitecturas de microservicios, lo que genera el problema de que las aplicaciones de arquitectura de microservicios son difíciles de probar y evaluar (Gong y Cai, 2023, p. 1).

La dificultad de las pruebas de un extremo a otro es la creación y el mantenimiento del entorno de pruebas. Es necesario implementar por completo una gran aplicación de arquitectura de microservicios, lo que consume muchos recursos. La restauración del medio ambiente después de las pruebas también requiere mucho tiempo y trabajo. Por lo tanto, los aspectos más importantes a considerar en las

pruebas de un extremo a otro son la integración continua, la implementación continua y las pruebas automatizadas (Gong y Cai, 2023, pp. 4-5).

Debido a las diferencias estructurales entre las aplicaciones de microservicios y las aplicaciones monolíticas tradicionales, la relación de dependencia entre los microservicios se está volviendo cada vez más compleja y es necesario actualizar la tecnología de prueba para las aplicaciones de microservicios (Lin et al., 2020, p. 1).

Trabajos de investigación recientes indican que las pruebas de seguridad de los microservicios a menudo se descuidan por motivos como la falta de tiempo, la falta de experiencia en el ámbito de la seguridad y la ausencia de entornos de prueba automatizados. A pesar de sus numerosos beneficios, existen muchos desafíos de seguridad en la arquitectura de microservicios. Estos desafíos incluyen una superficie de ataque ampliada, falta de capacidades de monitoreo en múltiples nubes y rastreo de seguridad distribuido [4]; Hay más puntos de entrada en los microservicios en comparación con las aplicaciones monolíticas. Las soluciones de seguridad heredadas que son efectivas para aplicaciones monolíticas a menudo no son lo suficientemente buenas para aplicaciones basadas en microservicios. También existe una ausencia de marcos de pruebas de seguridad totalmente automatizados para entornos de desarrollo ágiles (Ünver and Britto, 2023, pp. 1-2).

## 5.6 Amenazas a la validez

De acuerdo con Kitchenham et al. (2015), las limitaciones de la revisión son propias del contexto de la investigación las cuales no han sido o no pueden ser atendidas por el diseño de esta. Es así como, este apartado tiene la función de identificar las limitaciones que signifiquen una amenaza a la validez de los resultados de esta revisión. La Tabla 10 presenta las amenazas a la validez de la revisión.

***Tabla 10***

*Riesgos para la validez de la revisión sistemática de la literatura (RSL)*

| Identificador | Amenaza |
|---|---|
| $T$-1 | La restricción de acceso a estudios por parte del convenio entre el Consorcio Nacional de Recursos de Información Científica y Tecnológica (CONRICyT) y la Universidad Veracruzana. |
| $T$-2 | El sesgo inducido por la cantidad de estudios relevantes seleccionados manualmente ($T$). Dado que pudo afectar los resultados de la selección automatizada. |

# 6 Conclusiones

Este trabajo tiene como objetivo recopilar información referentes a pruebas aplicadas a sistemas basados en la arquitectura de microservicio. Ofreciendo la identificación de las pruebas, propuestas de la literatura (estado del arte), herramientas ampliamente utilizadas y retos por resolver para el estilo arquitectónico. Esto con la intención de proveer información relevante tanto a investigadores como practicantes interesados.

Después de analizar la información extraída y en contraste con las preguntas de investigación propuestas en esta revisión ($RQ$). Se aprecia que existe una considerable preocupación en la academia por solucionar problemas relacionados a la trazabilidad de defectos y la detección de fallos. Principalmente por la naturaleza y los mecanismos de comunicación entre servicios. Del mismo modo, se han propuesto una variedad de herramientas que atienden varios de los problemas de las herramientas tradicionales utilizadas en la industria.

En cuanto a las pruebas orientadas a los microservicios, las pruebas enfocadas a la calidad de lo servicios es un campo de investigación valioso. La gran mayoría de los trabajos de investigación centrados en este aspecto se enfoca en aspectos como el rendimiento, fiabilidad y escalabilidad de los microservicios.

Finalmente, en cuanto a los retos, se requieren propuestas en torno a modelos o metodologías de prueba aplicadas a microservicios, ya que aquellas tradicionales no satisfacen los requisitos introducidos por los microservicios.

# 7 Bibliografía


Alshuqayran, N., Ali, N. y Evans, R. (2016). A Systematic Mapping Study in Microservice Architecture. *2016 IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA)*, Macau, China, 44-51. https://doi.org/10.1109/SOCA.2016.15

Bass, L., Clements, P. y Kazman, R. (2015). Introduction. *Software Architecture in Practice* (3.ª ed.) (pp. 1-23). Pearson Education.

Bogner, J., Fritzsch, J., Wagner, S. y Zimmermann, A. (2021). Industry practices and challenges for the evolvability assurance of microservices. *Empirical Software Engineering*, *26*(104), 1-39. https://doi.org/10.1007/s10664-021-09999-9

Clemson, T. (2014, 18 de noviembre). *Testing Strategies in a Microservice Architecture* [presentación de diapositivas]. Martin Fowler. https://martinfowler.com/articles/microservice-testing/

Dijkstra, E. W. (1972). The humble programmer. *Communications of the ACM*, *15*(10), 859–866. https://doi.org/10.1145/355604.361591

Ding, H., Cheng, L. y Li, Q. (2020). An Automatic Test Data Generation Method for Microservice Application. *2020 International Conference on Computer Engineering and Application (ICCEA)*, Guangzhou, China, 188-191. https://doi.org/10.1109/ICCEA50009.2020.00048

Elsner, D., Bertagnolli, D., Pretschner, A. y Klaus, R. (2022). Challenges in regression test selection for end-to-end testing of microservice-based software systems. *In Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test (AST '22)*, Nueva York, NY, EUA, 1-5. https://doi.org/10.1145/3524481.3527217

Fowler, M. (2003). Who Needs an Architect? *IEEE Software*, *20*(5), 11-13. https://doi.org/10.1109/MS.2003.1231144

Fowler, M. y Lewis, J. (2014, 25 de marzo). *Microservices*. Martin Fowler. https://martinfowler.com/articles/microservices.html

Francesco, P. D., Malavolta, I. y Lago, P. (2017). Research on Architecting Microservices: Trends, Focus, and Potential for Industrial Adoption. *2017 IEEE International Conference on Software Architecture (ICSA)*, Gothenburg, Suecia, 21-30. https://doi.org/10.1109/ICSA.2017.24

Gong, J. y Cai, L. (2023). Analysis for Microservice Architecture Application Quality Model and Testing Method. *2023 26th ACIS International Winter Conference on Software Engineering, Artificial Intelligence, Networking and*

*Parallel/Distributed Computing (SNPD-Winter)*, Taiyuan, Taiwan, 141-145. https://doi.org/10.1109/SNPD-Winter57765.2023.10223960

Institute of Electrical and Electronics Engineers. (2017). *IEEE Standard for System, Software, and Hardware Verification and Validation* (IEEE Std 1012-2016). https://doi.org/10.1109/IEEESTD.2017.8055462

International Organization for Standardization, International Electrotechnical Commission e Institute of Electrical and Electronics Engineers. (2022). *ISO/IEC/IEEE International Standard - Software and systems engineering -- Software testing --Part 1: General concepts* (ISO/IEC/IEEE 29119-1:2022[E]). https://doi.org/10.1109/IEEESTD.2022.9698145

International Software Testing Qualifications Board. (2023). *Certified Tester: Foundation Level Syllabus (v4.0)*. https://istqb-main-web-prod.s3.amazonaws.com/media/documents/ISTQB_CTFL_Syllabus-v4.0.pdf

Jiang, P., Shen, Y. y Dai, Y. (2022). Efficient software test management system based on microservice architecture. *2022 IEEE 10th Joint International Information Technology and Artificial Intelligence Conference (ITAIC)*, Chongqing, China, 2339-2343. https://doi.org/10.1109/ITAIC54216.2022.9836605

Kalubowila, D. C., Athukorala, S. M., Tharaka, B. A. S., Samarasekara, H. W. Y. R., Samaratunge-Arachchilage, U. S. S. y Kasthurirathna, D. (2021). Optimization of Microservices Security. *2021 3rd International Conference on Advancements in Computing (ICAC)*, Colombo, Sri Lanka, 49-54. https://doi.org/10.1109/ICAC54203.2021.9671131

Kitchenham, B., Budgen, D. y Brereton, P. (2015). *Evidence-Based Software Engineering and Systematic Reviews*. Chapman & Hall/CRC.

Larrucea, X., Santamaria, I., Colomo-Palacios, R. y Ebert, C. (2018). Microservices. *IEEE Software*, *35*(3), 96-100. https://doi.org/10.1109/MS.2018.2141030

Li, H., Wang, J., Dai, H. y Lv, B. (2020). Research on Microservice Application Testing System. *2020 IEEE 3rd International Conference on Information Systems and Computer Aided Education (ICISCAE)*, Dalian, China, 363-368. https://doi.org/10.1109/ICISCAE51034.2020.9236829

Newman, S. (2015). Testing. *Building Microservices* (1.ª ed.) (pp. 131-154). O'Reilly Media.

Popay, J., Roberts, H., Sowden, A., Petticrew, M., Arai, L., Rodgers, M., Britten, N., Roen, K. y Duffy, S. (2006). *Guidance on the conduct of narrative synthesis in systematic reviews: A product from the ESRC Methods Programme*. https://doi.org/10.13140/2.1.1018.4643

Richards, M. y Ford, N. (2021a). Microservices Architecture. *Fundamentals of Software Architecture* (1.ª ed.) (pp. 245-265). O'Reilly Media.

Richards, M. y Ford, N. (2021b). Architectural Styles. *Fundamentals of Software Architecture* (1.ª ed.) (pp. 117-141). O'Reilly Media.

Rodgers, M., Sowden, A., Petticrew, M., Arai, L., Roberts, H., Britten, N., y Popay, J. (2009). Testing Methodological Guidance on the Conduct of Narrative Synthesis in Systematic Reviews: Effectiveness of Interventions to Promote Smoke Alarm Ownership and Function. *Evaluation*, *15*(1), 49-73. https://doi.org/10.1177/1356389008097871

Sommerville, I. (2016). Software testing. *Software Engineering* (10.ª ed.) (pp. 226-254). Pearson Education.

Thönes, J. (2015). Microservices. *IEEE Software*, *32*(1), 113-116. https://doi.org/10.1109/MS.2015.11

Wang, Z., Xia, Y., Sun, C. y Cheng, L. (2020). Research on microservice application performance monitoring framework and elastic scaling mode. *Journal of Physics: Conference Series*, *1617*(1), 1-7. https://doi.org/10.1088/1742-6596/1617/1/012048

Waseem, M., Liang, P., Márquez, G. y Salle, A. D. (2020). Testing Microservices Architecture-Based Applications: A Systematic Mapping Study. *2020 27th Asia-Pacific Software Engineering Conference (APSEC)*, Singapur, Singapur, 119-128. https://doi.org/10.1109/APSEC51365.2020.00020

Wu, C. F., Ma, S. P., Shau, A. C. y Yeh, H. W. (2022). Testing for Event-Driven Microservices Based on Consumer-Driven Contracts and State Models. *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*, Japan, 467-471. https://doi.org/10.1109/APSEC57359.2022.00064

Zhang, H., Babar, M. A., & Tell, P. (2011). Identifying relevant studies in software engineering. *Information and Software Technology*, *53*(6), 625–637. https://doi.org/10.1016/j.infsof.2010.12.010

Zhou, X., Peng, X., Xie, T., Sun, J., Ji, C., Li, W. y Ding, D. (2018). Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study. *IEEE Transactions on Software Engineering*, *47*(2), 243-260. https://doi.org/10.1109/TSE.2018.2887384

# 8 Referencias


Amaral, M., Chiba, T., Trent, S., Yoshimura, T. y Choochotkaew, S. (2022). MicroLens: A Performance Analysis Framework for Microservices Using Hidden Metrics With BPF. *2022 IEEE 15th International Conference on Cloud Computing (CLOUD)*, Barcelona, España, 230–240. https://doi.org/10.1109/CLOUD55607.2022.00043

Bi, T., Pan, Y., Jiang, X., Ma, M. y Wang, P. (2022). VECROsim: A Versatile Metric-oriented Microservice Fault Simulation System (Tools and Artifact Track). *2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE)*, Charlotte, NC, Estados Unidos, 297–308. https://doi.org/10.1109/ISSRE55969.2022.00037

Camilli, M., Guerriero, A., Janes, A., Russo, B. y Russo, S. (2022). Microservices integrated performance and reliability testing. *Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test (AST '22)*, Nueva York, NY, EUA, 29-39. https://doi.org/10.1145/3524481.3527233

Carroll, J. J., Anand, P. y Guo, D. (2021). Preproduction Deploys: Cloud-Native Integration Testing. *2021 IEEE Cloud Summit (Cloud Summit)*, Hempstead, NY, Estados Unidos, 41–48. https://doi.org/10.1109/IEEECloudSummit52029.2021.00015

Chen, H., Chen, P. y Yu, G. (2020). A Framework of Virtual War Room and Matrix Sketch-Based Streaming Anomaly Detection for Microservice Systems. *IEEE Access*, *8*, 43413–43426. https://doi.org/10.1109/ACCESS.2020.2977464

Chen, L., Wu, J., Yang, H. y Zhang, K. (2023). A microservice regression testing selection approach based on belief propagation. *Journal of Cloud Computing*, *12*(1), 20. https://doi.org/10.1186/s13677-023-00398-7

De Angelis, E., De Angelis, G., Pellegrini, A. y Proietti, M. (2021). Inferring Relations Among Test Programs in Microservices Applications. *2021 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, Oxford, Reino Unido, 114–123. https://doi.org/10.1109/SOSE52839.2021.00018

Detti, A., Funari, L. y Petrucci, L. (2023). μBench: An Open-Source Factory of Benchmark Microservice Applications. *IEEE Transactions on Parallel and Distributed Systems*, *34*(3), 968–980. https://doi.org/10.1109/TPDS.2023.3236447

Ding, H., Cheng, L. y Li, Q. (2020). An Automatic Test Data Generation Method for Microservice Application. *2020 International Conference on Computer Engineering and Application (ICCEA)*, Guangzhou, China, 188-191. https://doi.org/10.1109/ICCEA50009.2020.00048

Duan, T., Li, D., Xuan, J., Du, F., Li, J., Du, J. y Wu, S. (2021). Design and Implementation of Intelligent Automated Testing of Microservice Application. *2021 IEEE 5th Information Technology, Networking, Electronic and Automation Control Conference (ITNEC)*, Xi'an, China, 1306-1309. https://doi.org/10.1109/ITNEC52019.2021.9587260

Eismann, S., Bezemer, C. P., Shang, W., Okanović, D. y van Hoorn, A. (2020). Microservices: A Performance Tester's Dream or Nightmare? *Proceedings of the ACM/SPEC International Conference on Performance Engineering*, Nueva York, NY, Estados Unidos, 138–149. https://doi.org/10.1145/3358960.3379124

Elsner, D., Bertagnolli, D., Pretschner, A. y Klaus, R. (2022). Challenges in regression test selection for end-to-end testing of microservice-based software systems. *Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test (AST '22)*, Nueva York, NY, EUA, 1-5. https://doi.org/10.1145/3524481.3527217

Gartziandia, A. (2021). Microservice-Based Performance Problem Detection in Cyber-Physical System Software Updates. *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, Madrid, España, 147–149. https://doi.org/10.1109/ICSE-Companion52605.2021.00062

Gazzola, L., Goldstein, M., Mariani, L., Segall, I. y Ussi, L. (2020). Automatic Ex-Vivo Regression Testing of Microservices. *2020 IEEE/ACM 15th International Conference on Automation of Software Test (AST)*, Seúl, Corea del Sur, 11–20. https://ieeexplore.ieee.org/document/10186483

Giamattei, L., Guerriero, A., Pietrantuono, R. y Russo, S. (2022). Automated Grey-Box Testing of Microservice Architectures. *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*, Guangzhou, China, 640–650. https://doi.org/10.1109/QRS57517.2022.00070

Gong, J. y Cai, L. (2023). Analysis for Microservice Architecture Application Quality Model and Testing Method. *2023 26th ACIS International Winter Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD-Winter)*, Taiyuan, Taiwan, 141-145. https://doi.org/10.1109/SNPD-Winter57765.2023.10223960

Jin, M., Lv, A., Zhu, Y., Wen, Z., Zhong, Y., Zhao, Z., Wu, J., Li, H., He, H. y Chen, F. (2020). An Anomaly Detection Algorithm for Microservice Architecture Based on Robust Principal Component Analysis. *IEEE Access*, *8*, 226397–226408. https://doi.org/10.1109/ACCESS.2020.3044610

Kargar, M. J. y Hanifizade, A. (2018). Automation of regression test in microservice architecture. *2018 4th International Conference on Web Research (ICWR)*, Tehrán, Irán, 133–137. https://doi.org/10.1109/ICWR.2018.8387249

Karn, R. R., Das, R., Pant, D. R., Heikkonen, J., & Kanth, R. (2022). Automated Testing and Resilience of Microservice's Network-link using Istio Service Mesh. 2022 31st Conference of Open Innovations Association (FRUCT), 79–88. https://doi.org/10.23919/FRUCT54823.2022.9770890

Kohyarnejadfard, I., Aloise, D., Azhari, S. V. y Dagenais, M. R. (2022). Anomaly detection in microservice environments using distributed tracing data analysis and NLP. *Journal of Cloud Computing*, *11*(1), 25. https://doi.org/10.1186/s13677-022-00296-4

Lei, Q., Liao, W., Jiang, Y., Yang, M. y Li, H. (2019). Performance and Scalability Testing Strategy Based on Kubemark. *2019 IEEE 4th International Conference on Cloud Computing and Big Data Analysis (ICCCBDA)*, Chengdu, China, 511–516. https://doi.org/10.1109/ICCCBDA.2019.8725658

Li, H., Wang, J., Dai, H. y Lv, B. (2020). Research on Microservice Application Testing System. *2020 IEEE 3rd International Conference on Information Systems and Computer Aided Education (ICISCAE)*, Dalian, China, 363–368. https://doi.org/10.1109/ICISCAE51034.2020.9236829

Li, Z., Chen, J., Jiao, R., Zhao, N., Wang, Z., Zhang, S., Wu, Y., Jiang, L., Yan, L., Wang, Z., Chen, Z., Zhang, W., Nie, X., Sui, K. y Pei, D. (2021). Practical Root Cause Localization for Microservice Systems via Trace Analysis. *2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS)*, Tokio, Japón, 1–10. https://doi.org/10.1109/IWQOS52092.2021.9521340

Lin, D., Liping, F., Jiajia, H., Qingzhao, T., Changhua, S. y Xiaohui, Z. (2020). Research on Microservice Application Testing Based on Mock Technology. *2020 International Conference on Virtual Reality and Intelligent Systems (ICVRIS)*, Zhangjiajie, China, 815–819. https://doi.org/10.1109/ICVRIS51417.2020.00200

Long, Z., Wu, G., Chen, X., Cui, C., Chen, W. y Wei, J. (2020). Fitness-guided Resilience Testing of Microservice-based Applications. *2020 IEEE International Conference on Web Services (ICWS)*, Beijing, China, 151–158. https://doi.org/10.1109/ICWS49710.2020.00027

Ma, S. P., Yang, Y. Y., Lee, S. J. y Yeh, H. W. (2023). UTEMS: A Unit Testing Scheme for Event-driven Microservices. *2023 10th International Conference on Dependable Systems and Their Applications (DSA)*, Tokio, Japón, 591–592. https://doi.org/10.1109/DSA59317.2023.00085

Martin, J. M. R. (2021). Designing and Verifying Microservices Using CSP. *2021 IEEE Concurrent Processes Architectures and Embedded Systems Virtual Conference (COPA)*, San Diego, CA, Estados Unidos, 1–4. https://doi.org/10.1109/COPA51043.2021.9541471

Maulana, A. y Raharja, P. A. (2022). Design and Testing on Migration of Remiss-Supply in Banking System to Microservice Architecture. *2022 IEEE International*

*Conference on Communication, Networks and Satellite (COMNETSAT)*, Solo, Indonesia, 168–173. https://doi.org/10.1109/COMNETSAT56033.2022.9994362

Meng, X, Duan, X., Tao, W., Luan, Y., Zhang, J. y Wu, D. (2021). Modeling and verification of Industrial Microservice Architecture Based on Formal Methods. *2021 China Automation Congress (CAC)*, Beijing, China, 3776-3780. https://doi.org/10.1109/CAC53003.2021.9727653

Mohammad, R. A. y Kistijantoro, A. I. (2022). Development of Performance Regression Analysis Tool using Distributed Tracing on microservice-based Application. *2022 9th International Conference on Advanced Informatics: Concepts, Theory and Applications (ICAICTA)*, Tokoname, Japón, 1–6. https://doi.org/10.1109/ICAICTA56449.2022.9932918

Pei, Q., Liang, Z., Wang, Z., Cui, L., Long, Z. y Wu, G. (2023). Search-Based Performance Testing and Analysis for Microservice-Based Digital Power Applications. *2023 6th International Conference on Energy, Electrical and Power Engineering (CEEPE)*, Guangzhou, China, 1522–1527. https://doi.org/10.1109/CEEPE58418.2023.10165808

Pietrantuono, R., Russo, S. y Guerriero, A. (2018). Run-Time Reliability Estimation of Microservice Architectures. *2018 IEEE 29th International Symposium on Software Reliability Engineering (ISSRE)*, 25–35. https://doi.org/10.1109/ISSRE.2018.00014

Pina, F., Correia, J., Filipe, R., Araujo, F. y Cardroom, J. (2018). Nonintrusive Monitoring of Microservice-Based Systems. *2018 IEEE 17th International Symposium on Network Computing and Applications (NCA)*, Cambridge, MA, Estados Unidos, 1–8. https://doi.org/10.1109/NCA.2018.8548311

Quenum, J. G. y Aknine, S. (2018). Towards Executable Specifications for Microservices. *2018 IEEE International Conference on Services Computing (SCC)*, San Francisco, CA, Estados Unidos, 41–48. https://doi.org/10.1109/SCC.2018.00013

Quin, F. y Weyns, D. (2022). SEAByTE: A Self-adaptive Micro-service System Artifact for Automating A/B Testing. *2022 International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, Nueva York, NY, Estados Unidos, 77–83. https://doi.org/10.1145/3524844.3528081

Rangaiyengar, R., Komondoor, R. y Medicherla, R. K. (2023). Multi-Layer Observability for Fault Localization in Microservices Based Systems. *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, Taipa, Macao, 733–737. https://doi.org/10.1109/SANER56733.2023.00081

Rattanukul, P., Makaranond, C., Watanakulcharus, P., Ragkhitwetsagul, C., Nearunchorn, T., Visoottiviseth, V., Choetkiertikul, M. y Sunetnanta, T. (2023). Microusity: A testing tool for Backends for Frontends (BFF) Microservice Systems.

*2023 IEEE/ACM 31st International Conference on Program Comprehension (ICPC)*, Melbourne, Australia, 74–78. https://doi.org/10.1109/ICPC58990.2023.00021

Reile, C., Chadha, M., Hauner, V., Jindal, A., Hofmann, B. y Gerndt, M. (2022). Bunk8s: Enabling Easy Integration Testing of Microservices in Kubernetes. *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, Honolulu, HI, Estados Unidos, 459–463. https://doi.org/10.1109/SANER53432.2022.00062

Schulz, H., Angerstein, T., Okanović, D. y van Hoorn, A. (2019). Microservice-Tailored Generation of Session-Based Workload Models for Representative Load Testing. *2019 IEEE 27th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, Rennes, Francia, 323–335. https://doi.org/10.1109/MASCOTS.2019.00043

Sedghpour, M. R. S., Duque, A. O., Cai, X., Skubic, B., Elmroth, E., Klein, C. y Tordsson, J. (2023). HydraGen: A Microservice Benchmark Generator. *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*, Chicago, IL, Estados Unidos, 189–200. https://doi.org/10.1109/CLOUD60044.2023.00030

Smith, S., Robinson, E., Frederiksen, T., Stevens, T., Cerny, T., Bures, M. y Taibi, D. (2023). Benchmarks for End-to-End Microservices Testing. *2023 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, Atenas, Grecia, 60–66. https://doi.org/10.1109/SOSE58276.2023.00013

Sotomayor, J. P., Allala, S. C., Alt, P., Phillips, J., King, T. M. y Clarke, P. J. (2019). Comparison of Runtime Testing Tools for Microservices. *2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC)*, Milwaukee, WI, Estados Unidos, 356–361. https://doi.org/10.1109/COMPSAC.2019.10232

Ünver, B. y Britto, R. (2023). Automatic Detection of Security Deficiencies and Refactoring Advises for Microservices. *2023 IEEE/ACM International Conference on Software and System Processes (ICSSP)*, Melbourne, Australia, 25–34. https://doi.org/10.1109/ICSSP59042.2023.00013

van Hoorn, A., Aleti, A., Düllmann, T. F. y Pitakrat, T. (2018). ORCAS: Efficient Resilience Benchmarking of Microservice Architectures. *2018 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, Memphis, TN, Estados Unidos, 146–147. https://doi.org/10.1109/ISSREW.2018.00-10

Vassiliou-Gioles, T. (2020). A simple, lightweight framework for testing RESTful services with TTCN-3. *2020 IEEE 20th International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, Macau, China, 498–505. https://doi.org/10.1109/QRS-C51114.2020.00089

Vassiliou-Gioles, T. (2021). Quality Assurance of Micro-Services—When to trust your micro-service test results? *2021 IEEE 21st International Conference on*

*Software Quality, Reliability and Security Companion (QRS-C)*, Hainan, China, 1–6. https://doi.org/10.1109/QRS-C55045.2021.00024

Wang, L., Zhao, N., Chen, J., Li, P., Zhang, W. y Sui, K. (2020). Root-Cause Metric Location for Microservice Systems via Log Anomaly Detection. *2020 IEEE International Conference on Web Services (ICWS)*, Beijing, China, 142–150. https://doi.org/10.1109/ICWS49710.2020.00026

Wang, Y., Cheng, L. y Sun, X. (2019). Design and Research of Microservice Application Automation Testing Framework. 2019 International Conference on Information Technology and Computer Application (ITCA), Guangzhou, China, 257–260. https://doi.org/10.1109/ITCA49981.2019.00063

Wu, C. F., Ma, S. P., Shau, A. C. y Yeh, H. W. (2022). Testing for Event-Driven Microservices Based on Consumer-Driven Contracts and State Models. *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*, Japón, 467–471. https://doi.org/10.1109/APSEC57359.2022.00064

Wu, L., Tordsson, J., Bogatinovski, J., Elmroth, E. y Kao, O. (2021). MicroDiag: Fine-grained Performance Diagnosis for Microservice Systems. *2021 IEEE/ACM International Workshop on Cloud Intelligence (CloudIntelligence)*, Madrid, España, 31–36. https://doi.org/10.1109/CloudIntelligence52565.2021.00015

Xie, R., Yang, J., Li, J. y Wang, L. (2023). ImpactTracer: Root Cause Localization in Microservices Based on Fault Propagation Modeling. *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, Antwerp, Bélgica, 1–6. https://doi.org/10.23919/DATE56975.2023.10137078

Yang, J., Guo, Y., Chen, Y., Zhao, Y., Lu, Z. y Liang, Y. (2022). Robust Anomaly Diagnosis in Heterogeneous Microservices Systems under Variable Invocations. *GLOBECOM 2022 - 2022 IEEE Global Communications Conference*, Río de Janeiro, Brasil, 2722–2727. https://doi.org/10.1109/GLOBECOM48099.2022.10001270

# 9 Apéndices

**Tabla A1**

*Variantes de la cadena búsqueda seleccionada aplicadas en los motores de búsqueda elegidos*

| Motor de búsqueda | Cadena de búsqueda |
|---|---|
| IEEE Xplore | ((microservice* OR “microservice? architect*”) AND (test* OR verification OR validation)) |
| ACM Digital Library | ((microservice* OR “microservice? architect*”) AND (test* OR verification OR validation)) |
| SpringerLink | ((microservice* OR “microservice? architect*”) AND (test* OR verification OR validation)) |
| ScienceDirect | ((microservice OR "microservices architecture") AND (testing OR verification OR validation)) |